UnionPay Integrated Circuit Card Specifications
— Product Specifications

Part IX Multi-interface Card Specification

Version 2014

THIS PAGE IS INTENTIONALLY LEFT BLANK.

Table of Contents

Summary of Revisions

The change listed below is associated with the current version.

Description of Change	Where to look
Revised: Normative References, added "UnionPay Integrated Circuit Card Specifications - Product Specifications Part 16".	2
Added: "Audio Track" definition.	3.2
Removed: "Secure Module" definition.	3.7
Removed: "Virtual keyboards" definition.	3.8
Added: "Digital Certificate" definition.	3.8
Added: "Internet banking" definition.	3.9
Added: "Mobile banking" definition.	3.10
Added: "Anti-hijacking" definition.	3.11
Added: "Anti-hijacking threshold" definition.	3.12
Revised: Figure 1 Multi-interface card – minimum hardware module icons.	5.1.1
Revised: Security Main Control Modules	5.1.3
Removed: Main Control Modules	5.1.4
Revised: Minimum Hardware Components	5.1
Added: Reliability	5.2
Added: Transmission Speed	5.3
Added: Field Intensity Requirements	5.4
Revised: Product Morphology	6
Revised: Work Modes	7
Revised: Functions	8.1
Revised: Standard Debit/Credit Transaction Process	8.2.1
Added: Contactless Mode Low-value Payment Transaction Process	8.2.2
Added: Digital Certificate Authentication and Signature Process	8.2.3
Revised: Communication Interface	9.1
Revised: Updated the title to "USB Communication".	9.2
Added: Included physical definition, electric definition, and programming interface under "Audio Communication".	9.3
Revised: Transaction Anti-hijack Principle	10.4
Added: Audio Access Mode Exception Handling	10.6
Added: Digital Certificate Security	10.7
Added: Appendix F Audio Communication API	Appendix F
Added: Appendix G Anti-hijack Configuration	Appendix G

1 Application Scope

This book applies to all UPI participants.

2 Normative References

These standards reference the provisions of the following documents, and these provisions may therefore be considered to be provisions of the standards themselves. For all dated reference documents, all later amendments (except errata corrections) or modifications are not necessarily applicable to these standards. For this reason it is recommended that all parties associated with this agreement conduct all necessary research and come to an understanding regarding what newer versions of these documents remain applicable. For all non-dated documents, their most recent version may be considered to be applicable to these standards.

UnionPay Technical Specifications on Bankcard Interoperability V2.1

ISO 4208:1977 Detachable pilots for use with counterbores and 90 degrees countersinks -- Dimensions

ISO 4943:1985 Steel and cast iron -- Determination of copper content -- Flame atomic absorption spectrometric method

CISPR 22-2006 Information technology equipment— Radio disturbance characteristics— Limits and methods of measurement

ISO8583-1987 Financial transaction card originated messages -- Interchange message specifications

UnionPay Integrated Circuit Card Specifications - Basic Specifications Part 3

UnionPay Integrated Circuit Card Specifications - Product Specifications Part 16

Universal Serial Bus Specification Revision 2.0

Universal Serial Bus Micro-USB Cables and Connectors Specification Revision 1.01

Universal Serial Bus Specification for Integrated Circuit(s) Cards Interface Devices Revision 1.1

3 Terms and Definitions

The following terminology and definitions apply to these standards.

3.1 Audio Interface

Earphone and microphone interface equipment that communicates with smart devices and other devices by transmitting modulated analogue signals; an optional interface for multi-interface cards.

3.2 Audio Track

A communication channel in storage device used to collect and playback the audio during audio recording or playing. The number of audio track refers to the number of audio source during recording or corresponding number of loudspeaker during playback.

3.3 UnionPay Multi-Interface IC Card

Within these specifications, “UnionPay multi-interface IC cards”, “multi-interface financial IC cards”, and “multi-interface cards” all refer to contactless IC cards equipped with USB interface and other optional communication interface (such as audio interface); these cards support standard debit/credit function and contactless low-value payment function, and also support digital certificate functions.

3.4 Point of Entrance

Personal computers, smart devices, and other types of electronic devices with internet access and also capable of connecting with multi-interface cards via USB or other communication interface,

3.5 Processing Center

A system that receives processes and forwards transaction request messages from the multi-interface IC card and then returns the transaction result messages to the multi-interface card.

3.6 Secure Channel

A secure communication channel formed between a multi-interface card and a processing center.

3.7 Secure Channel Application

An application of the multi-interface IC card secure module that is responsible for establishing a secure channel with the processing center.

3.8 Digital Certificate

An asymmetric cryptographic transformation of data that allows the recipient of the data to prove the origin and integrity of the data, and protect the sender and the recipient of the data against forgery by third parties, and the sender against forgery by the recipient.

3.9 Digital Signature

Data existing in data information for demonstrating the authenticity of a digital message, which can be used for identifying the recipient.

3.10 Internet banking

Banking system that provide customer financial service via internet

3.11 Mobile banking

Banking system that provide customer financial service via mobile network

3.12 Anti-hijacking

During no display or keyboard contactless card transactions and digital certificate signature process, card or digital certificate device can be hijacked, further cause information leak and fraud. Multiple interface card use monitor to display transaction information and require cardholder to touch to confirm, in order to prevent transaction information from being hijacked.

3.13 Anti-hijacking threshold

Multiple interface card which has a minimal transaction amount below which does not require cardholder to touch to confirm transaction.

4 Symbols and Abbreviations

The following symbols and abbreviations are applicable to these standards:

CA	Certificate Authority
CCID	Chip/Smart Card Interface Devices
FIPS	Federal Information Processing Standard
HMAC	Keyed-Hash Message Authentication Code
PC	Personal Computer
RA	Register Authority
USB	Universal Serial BUS
X.509	IETF's PKIX Certificate

5 Hardware Requirements

5.1 Minimum Hardware Components

5.1.1 Module Icons

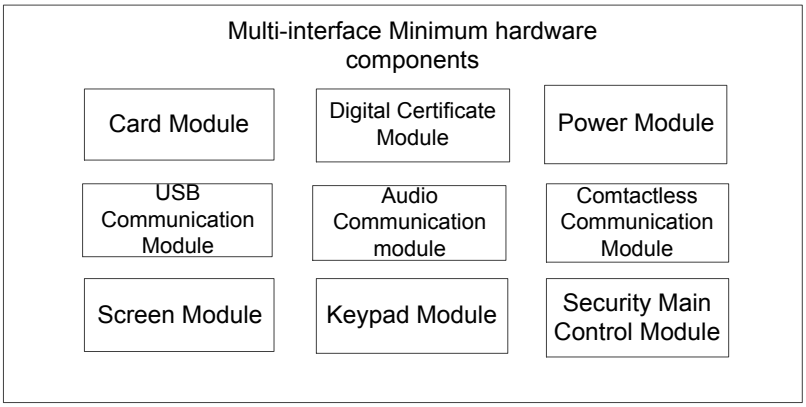


Figure 1 Multi-Interface Cards - Minimum Hardware Module Icons

5.1.2 Card Modules

Modules equipped with UICS Financial IC Card functionality that support contactless interfaces.

5.1.3 Security Main Control Modules

Module responsible for establishing a secure channel during multi-interface card transaction processes, providing secure key date store and encryption / decryption algorithm.

5.1.4 USB Communication Module

Communication module based on USB standards and in accordance with CCID protocols.

5.1.5 Audio Communication Module

Audio communication module is consisted by hardware code unit, and responsible for modulating and demodulating analogue signals. Audio interface can be part of multiple interface card or external connect with multiple interface card.

5.1.6 Power Module

Module used for power supply and power management.

5.1.7 Digital Certificate Module

Module that provide digital certificate functions.

5.1.8 Contactless Communication Module

Communication module based on contactless interface, comply with ISO14443 protocol. This module can be shutdown to prevent external malware attack.

5.1.9 Screen Module

Essential transaction information can be displayed to the cardholder by using additional screen modules; this allows the cardholder to immediately discover any potential transaction issues, thus increasing the security of transactions.

At the same time these screens can be used to display whether or not a transaction has been successful. This helps the user be intuitively knowledgeable of transaction information. By acting in concert with a keyboard, the cardholder can easily access account information such as electronic cash balance.

5.1.10 Keyboard Module

Keyboard modules can include important keys such as numeric key, cancel key, enter key, query key, and etc. Digital keyboards can be used to enter transaction PIN numbers; cancel buttons can be used by an user to immediately cancel a transaction; inquiry buttons can be used to inquire the offline electronic cash balance stored within a multi-interface card; Enter key can be used to confirm PIN entered by the user as well as to confirm the transaction details entered (such as the transaction amount, etc.).

5.2 Reliability

5.2.1 Magnet Reliability

Wireless magnet interference limit should comply with ISO standard.

5.2.2 Mean Time Between Failures

Except special components have special requirements, or MTBF should not less than 4000 hours.

5.2.3 Electrostatic Protection

UAB and audio interface should comply with ISO61000 standard.

5.2.4 Operating Temperature

Operating Temperature: -10℃~50℃

Storage Temperature: -20℃~70℃；

Operating Humidity: 0%~90%。

5.2.5 Structure

Shakeproof: can bear 1.5 meter height drop to mattess without structure broken

Waterproof: require to reach IP54 level.

Audio jacket life no less than 3000 times plugin and out (No load charge)

5.3 Transmission Speed

USB interface data transmission speed not less than 12Mb/s

Audio interface data transmission speed not less than 2kb/s

Contactless interface data transmission speed not less than 106kb/s

5.4 Field Intensity Requirement

Card module should be able to operate properly under field intensity between 1.5A/m to 7.5A/m.

6 Product Morphology

Multi-interface card must be equipped with contactless interface and USB. Optional expansions and other communication interfaces (such as audio interfaces) can be used to expand the functionality of multi-interface cards. This allows multi-interface cards to have a variety of forms.

Table 1 Product Morphology hardware requirements

Card module	Digital certificate module	Security main control module	Contactless communication module	USB communication module	Andio communication module	Power module	Screen module	Keypad
Required	Required	Required	Required	Required	Required	Required	Required	'ok' 'Cancel''up' 'down', optional 'power on' '0-9' '.'and more button

7 Work Modes

Multi-interface cards support three work modes: contactless mode, USB access mode and Audio access mode. Multi-interface cards can only use one work mode at a time; The priority of the two modes depends on which mode is used first to connect equipment.

- Contactless Card Mode

In this work mode, the multi-interface card is not inserted to any device and UICS financial transactions can only be conducted via the contactless interface.

- USB Access Mode

In this work mode, the multi-interface card has been connected to a device using a USB interface. Internet credit/debit transaction and digital certificate based authentication and digital signature is supported.

- Audio Access Mode

In this work mode, the multi-interface card has been connected to a device using an audio interface. Mobile network credit/debit transaction and digital certificate based authentication and digital signature is supported.

8 Functions and Processes

8.1 Functions

Multi-interface cards support UICS-standard debit/credit and contactless low-value payment functionalities, and also support digital certificate authentication functionality. Multi-interface cards should support anti-hijacking functions when processing above transactions. Detail information refers to chapter 10.

8.2 Processes

8.2.1 Standard Debit/Credit Transaction Process

When the multi-interface card is operating in contactless mode; please see *UICS basic specifications part.2* for standard debit/credit transaction process information.

Debit/credit transactions need support anti-hijacking, detail information refer to below diagram.

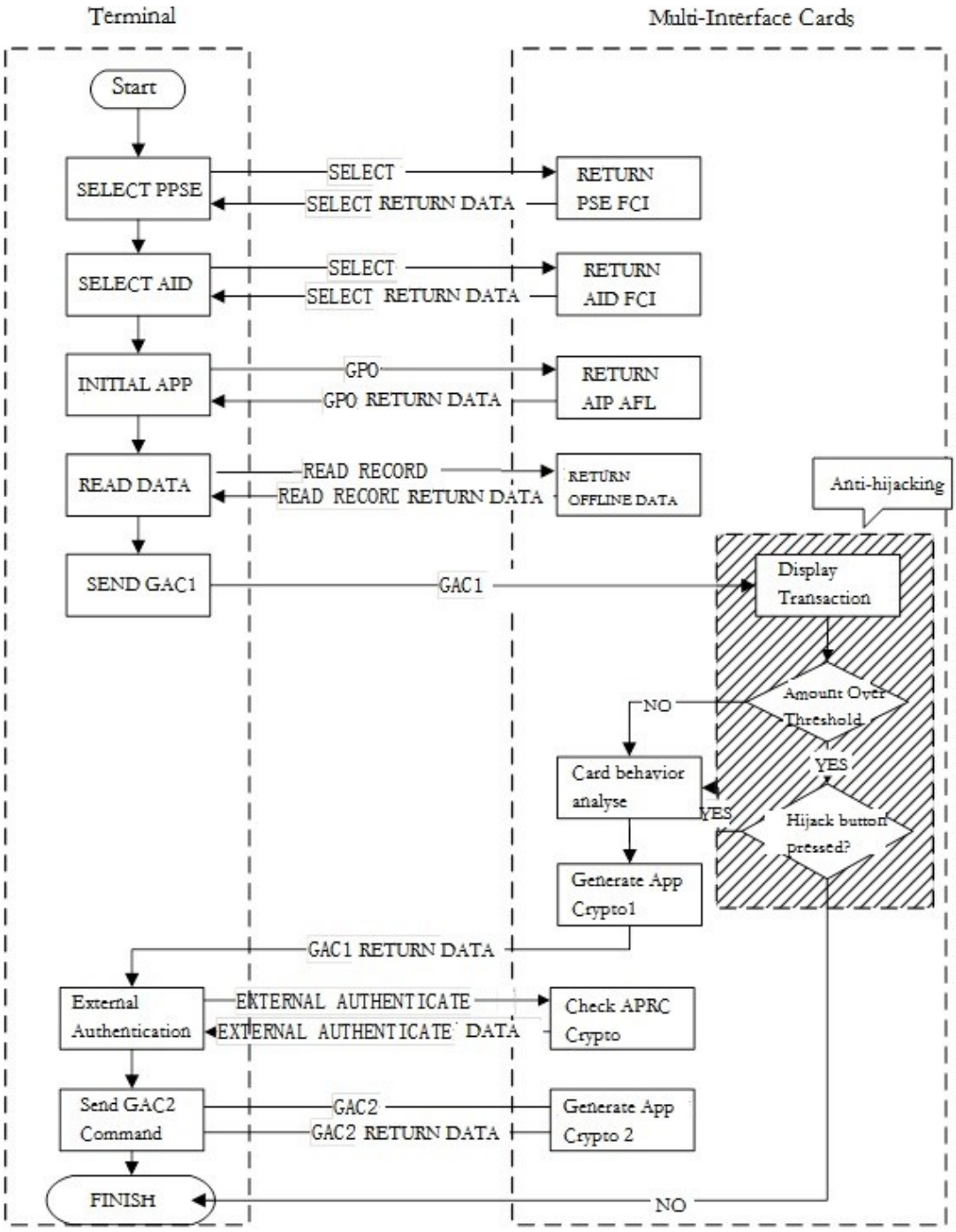


Figure 2 Multi-Interface Debit/Credit Transaction Process

When the multi-interface card is operating in device access mode, standard debit/credit transaction processes will occur as shown in the diagram below; the dotted line arrow indicates the corresponding action required security services provided by the secure channel application. See Chapter 11 for a description of secure channel applications.

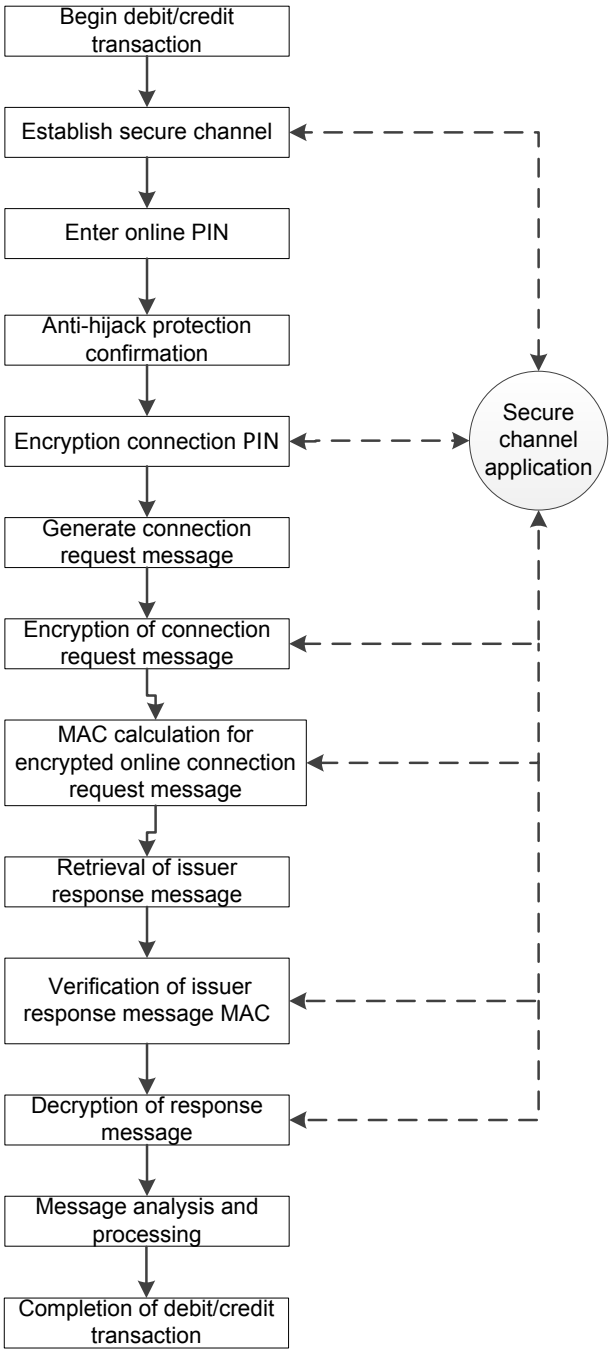


Figure 3 Device Access Mode Standard Debit/Credit Transaction Process Illustration

8.2.2 Contactless Mode Low-value Payment Transaction Process

When a multi-interface card is in contactless work mode micropayment transactions may be conducted. See *UICS Basic Specifications part 5* for transaction process details.

Low-value payment transaction should support anti-hijacking.

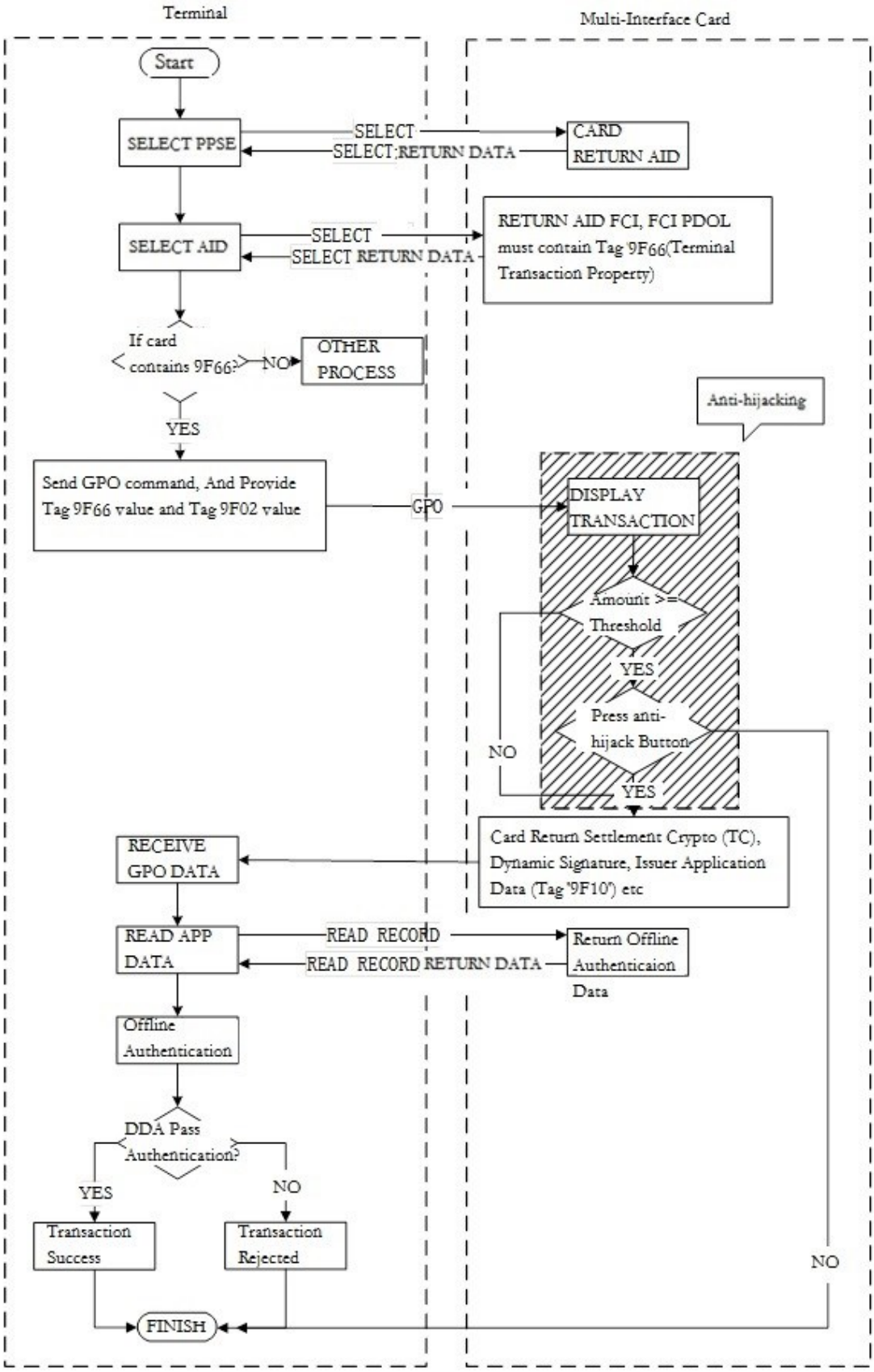


Figure 4 Multi-Interface Card Low-value payment Transaction Process

8.2.3 Digital Certificate Authentication and Signature Process

Digital certificate signature process should support anti-hijack.

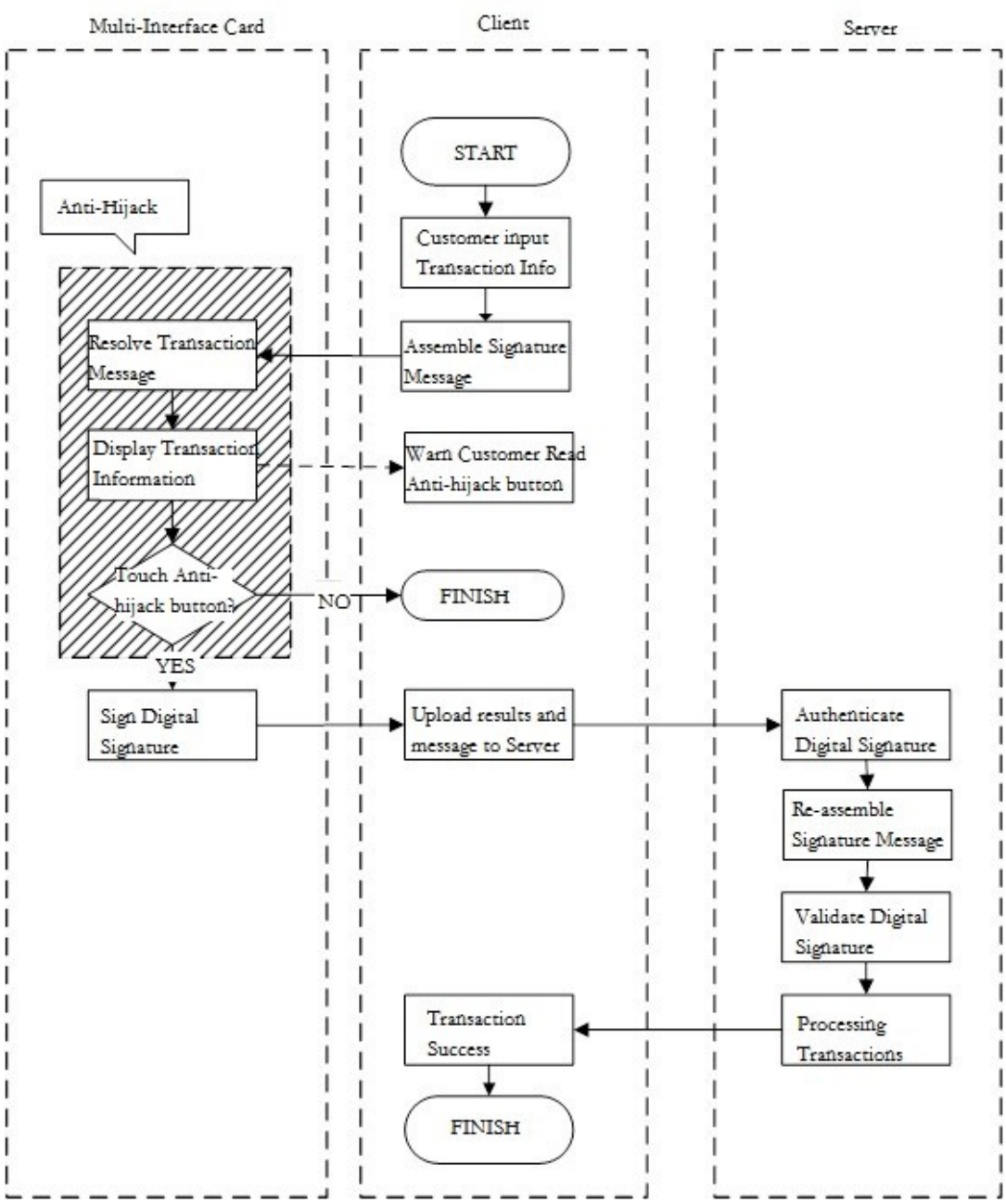


Figure 5 Multi-Interface Card Digital Signature Authentication and Sign Signature Process

9 Communication Interface

9.1 Communication Interface Hierarchy Graph

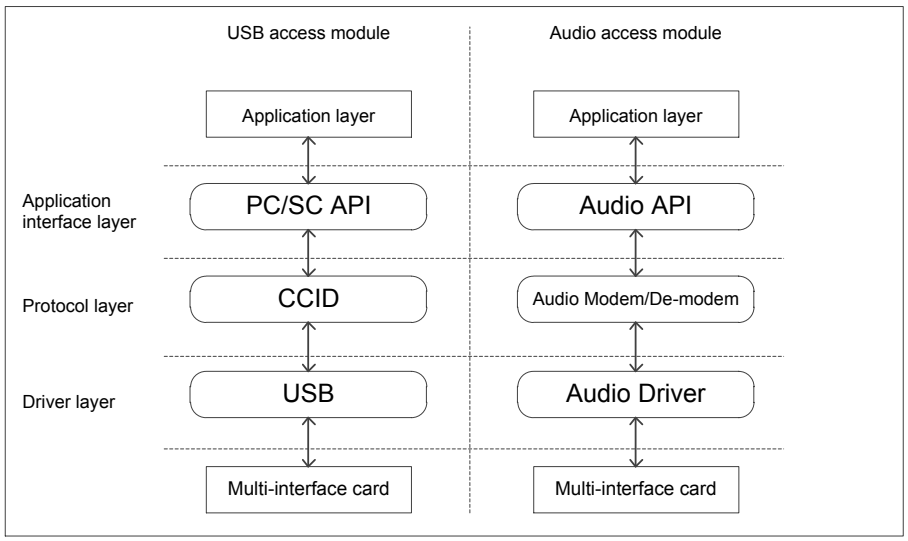


Figure 6 Communication Interface Hierarchy Graph

9.2 USB Communication

Communication based on USB interfaces are in accordance with both USB standards and CCID Protocols; the application layer interacts with the multi-interface card using a PC/SC programming interface.

Once a multi-interface card has accessed a device using a USB interface, the names of the multi-interface card devices will be enumerated. These names should be selected in accordance with the following rules:

'CUP_M'+ reserve character + '_' + vendor code + '_' + vendor type number

Note:

- '_' indicates a space;
- The reserve character is 1 byte in length and by default is the character "1";
- The vendor code and vendor type number strings must be expressed using English letters and numbers; the vendor code is a three-digit number and is distributed by UnionPay; the vendor type number must not be longer than 16 characters.

9.3 Audio Communication

9.3.1 Physical Definition

Use 3.5mm four chip plugin, Definition as below:

Standard Sequence 1: Left Track, Right Track, Micphone, Ground (Figure 7)

Standard Sequence 2: Left Track, Right Track, Ground, Micphone (Figure 7)

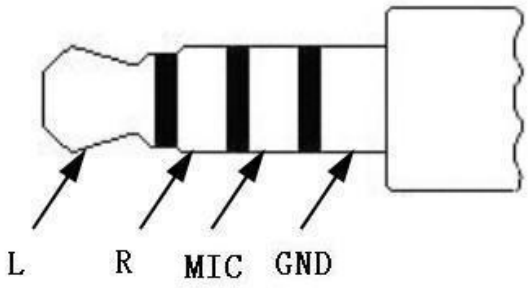


Figure 7 Audio Physical Standard

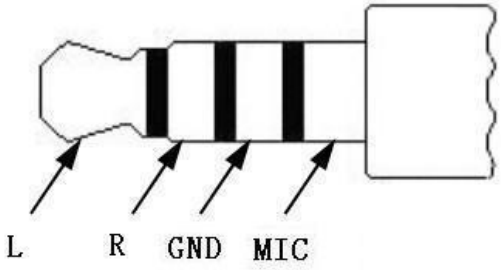


Figure 8 Audio Physical Standard

9.3.2 Electric Definition

Table 2 Electric Definition (From Smart Device to Multi-interface card)

Symbol	Definition	Min	Max
Vin	Audio track output voltage	-3.3V	+3.3V
Ilim	Audio track input limit electricity	NA	0.1A
Fsamp	Audio sampling frequency	44100HZ	96000HZ

Table 3 Electric Definition (From Multi-interface card to Smart Device)

Symbol	Definition	Min	Max
Vout	MIC output voltage	0.01V	+0.1V
Fsamp	Audio sampling frequency	8000HZ	48000HZ

9.3.3 Programming Interface

Appendix F.

10 Security System

10.1 Hardware Security Requirements

Internal security modules are required to conduct DES/RSA encryption algorithms and support a maximum of 2048-bit RSA calculations.

Defensive technology must be capable of effectively guarding against DPA, SPA, DFA, and other similar attack types.

In contactless work mode, hardware security must be in accordance with UICS *Basic Specifications part4* security requirements.

10.2 Certificate System

10.2.1 CA System Structure

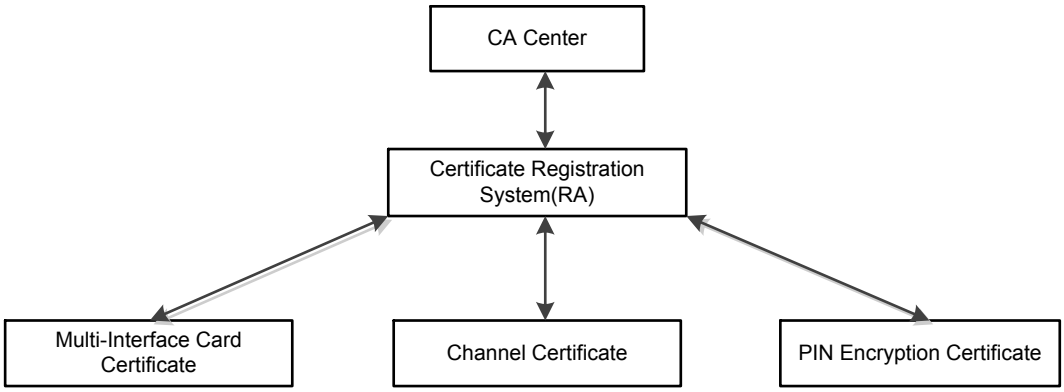


Figure 9 Multi-Interface Card CA System Structure

The CA Center is the upper most level of the certificate system structure and is responsible for signing certificate applications submitted by multi-interface cards and processing centers and generating certificates when appropriate.

The Certificate Registration System (RA) is generally used for auditing the certificate applications submitted by multi-interface card vendors and processing centers and supplying them with certificate downloads.

Multi-interface card certificates are submitted to CA for approval by the multi-interface card personalization institution. These certificates are in X.509 form and issued by the CA Center; they are used to identify the vendor that generated a given multi-interface card and use a public/private key pair generated internally by the multi-interface card. The private key of this private key pair is stored within the multi-interface card and is not exported.

Channel certificate are generated when an application is submitted to the CA by the processing center; certificates are then issued by the CA Center in X.509 form. Each channel server uses a unique channel certificate so as to verify the authenticity of the channel server and guard against fraudulent server activities.

PIN encryption certificates are generated when an application is submitted to the CA by the processing center; certificates are then issued by the CA Center in X.509 form and used to encrypt the cardholder's online PIN.

10.2.2 Key Algorithms

Symmetrical, assymetrical, and hash algorithms approved within *UICS Basic Specifications part 4*-are used.

10.3 Security Channel Principle

Multi-interface cards connect to the processing center by connecting to a device with internet access such as a PC and by utilizing handshake principles and a terminal-to-terminal logical secure channel created by the processing center. The structure of this connection is illustrated below.

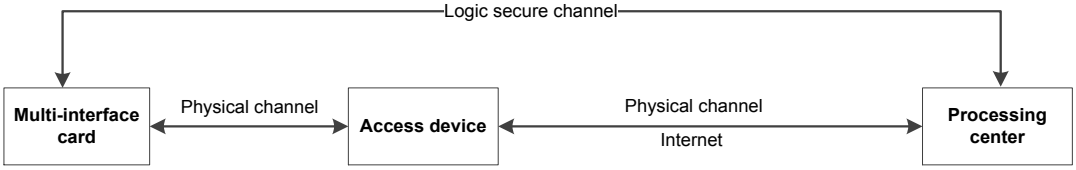


Figure 10 Secure Channel Diagram

During the establishment process of a local secure channel, access devices only transfer information between the multi-interface card and processing center; no other information processing occurs.

Secure channel establishment protocols are composed of handshake protocols and recording layer protocols. Handshake protocols are used for the cross-verification of multi-interface cards and the processing center and the transfer of dialogue keys. Recording layer protocols are used for the encrypted transmission of application data.

10.3.1 Handshake Protocol

Handshake protocols are used for the cross-verification of multi-interface cards and the processing center and the transfer of dialogue keys.

Handshake Protocol Basic Process

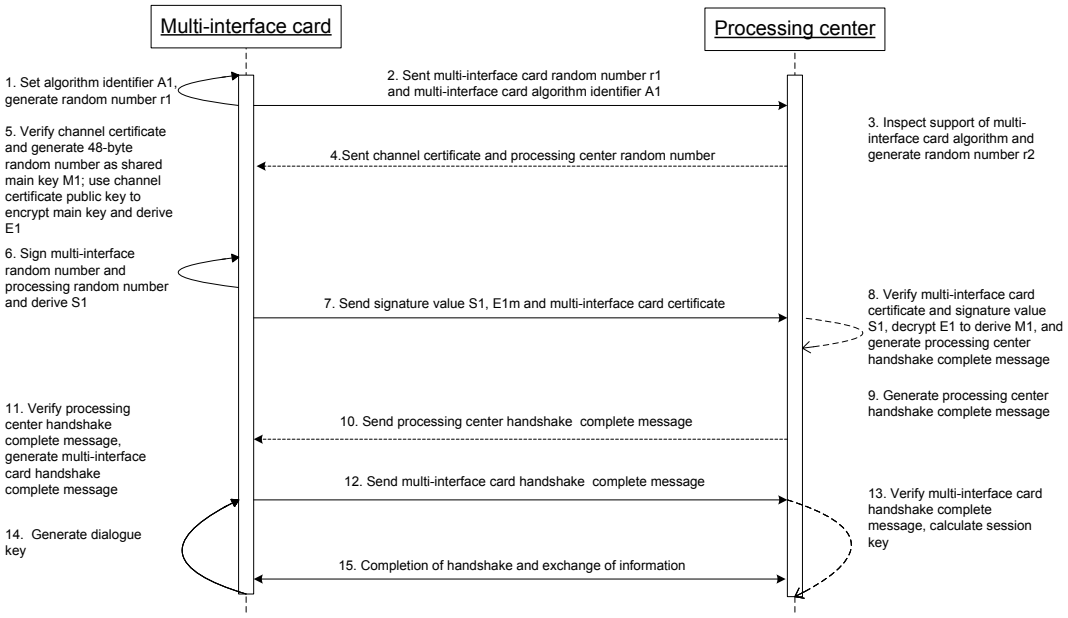


Figure 11 Handshake Protocol Information Sequence Diagram

Handshake Protocol Work Procedures

1) Multi-interface card acquires algorithm identifier A1 and generates random number r1; r1 and A1 are combined to produce R1; depending on the algorithm support settings of the multi-interface card, the following steps will require the usage of either symmetrical or asymetrical algorithms;

2) Multi-interface cards send random number and algorithm information to the processing center, thus initiating the handshake protocol;

3) The processing center selects algorithm identifier A2 and produced random number r2; r2 and A2 are combined to create R2. The processing center checks if it can support the algorithm information provided by the multi-interface card; if the processing center supports the algorithm then it will then initiate the encryption algorithm; if not supported the processing center will return an error message and break the connection;

4) The processing center sends a random number a processing center channel certificate;

5) The multi-interface card uses a preloaded CA Root certificate to verify the authenticity of the received processing center channel certificate; if the verification is not successful then an error message will be generated and the connection will be stopped; otherwise, the mutli-interface card will generate a 48-byte random number to act as a shared main key M1, the card will then use the assymetrical algorithm established before the usage of the public key provided by processing center channel certificate to encrypt M1 and generate E1;

6) R1 and R2 are connected to create R3; the mutli-interface card will use digest algorithm SHA-1 on R3 and create H1, then use its own private key to sign H1, thus creating S1;

7) The multi-interface card will then transmit S1, E1, and the multi-interface card certificate to the processing center;

8) The processing center uses the CA Root certificate to verify the legality of the mutli-interface card certificate; If the verification fails then an error message will be produced and the connection will be stopped; if verification succeeds then the mutli-interface card certificate will be used to verify S1. If S1 verification fails then then an error message will produced and the connection will be stopped; otherwise, E1 will be decrypted to yield shared main key M1;

9) The processing center will conduct SHA-1 operations on the channel certificate to obtain H2 and on the multi-interface card certificate to obtain H3. R1, R2, H2, H3, S1, and E1 are connected to obtain T1 (T1=R1||R2||H2||H3||S1||E1); SHA-1 will then be conducted on T1 to obtain H4; ASCII code “SERVER” and H4 are connected to yield D1; the first 16 bytes of M1 are used to conduct HMAC on D1 to obtain F1 (See Appendix B.3 for HMAC calculation algorithms);

10) The processing center sends verification of handshake completion F1 to the multi-interface card;

11) The multi-interface card will verify F1 sent from the processing center; if verification is not successful then then an error message will be produced and the connection will be stopped; if verification succeeds then the mutli-interface card will produce handshake verification message F2; F2 calculation is performed the same as that used for F1 but requires that the ASCII code “SERVER” obtained during F1 to be changed to ASCII code “CLIENT”;

12) The multi-interface card sends verification of handshake completion F2 to the processing center;

13) The processing center will use the same calculation process to verify F2. if verification is not successful then then an error message will be produced and the connection will be stopped;

14) Once the handshake procedure above is successful then both sites will use the following method to calculate a dialogue key:

X = HMAC(M1, key_label||R1||R2) (M1 uses its front 16 bytes)

Key-label is the 3-bite ASCII code "KEY". See Appendix B.3 for HMAC Calculation. If X1X2…X20 are the first twenty bytes of X, then an encryption key SKey is: SKey = X1X2…X16; MAC key MKey is: MKey = X5X6…X20;

15) Handshake completed.

10.3.2 Record Layer Protocol

Once the handshake has been completed then both parties will then establish a secure channel through which data is transmitted.

Record Layer Protocol Data Encryption Method

Before data is transmitted, a data block of length of 2-bytes should be added to the front, thus creating data block D = (Length||Data). Encryption key SKey is used to encrypt D in accordance with the encryption method agreed upon by the multi-interface card and processing center (See Appendix B.1), thus:

$EData = E_{SKey}(D)$;

Record Layer Protocol Data Integrity Protection Method

During record layer protocol transmission processing, all sent and received messages from both parties will be marked with a sequence number; the initial value Seq0 of this number is generated using the following method:

The first 8 bytes of multi-interface card random number Random1 are acquired as well as the first 8 bytes of processing center random number Random2, thus Seq0=Random1 || Random2.

For each record of a sent or received message, the record sequence number will add a 1, namely $Seq_i = Seq_{i-1} + 1$. Note that all parties must maintain sequence number synchronization.

The integrity of application data from both parties should be protected using information authentication code MAC; MAC is generated using the following method:

DataMAC = MAC(MKey, Seq_i || EData) (MKey uses its front 16 bytes)

EData is the transmitted encrypted application data; Seq is the current record sequence number. See Appendix B.2 for MAC calculation method. Once the multi-interface card or processing center has received data it will first verify MAC authenticity; if verified then processing will continue; if not, an error message will be generated. A verification error will interrupt the connection and the handshake protocol will need to be restarted,

10.4 Transaction Anti-Hijack Principle

Multi-interface card transaction anti-hijacking protection refers to forcing a cardholder go manual press an anti-hijack button while the multi-interface card is conducting an online transaction in device access mode; this confirms that the cardholder is aware of the transaction taking place.

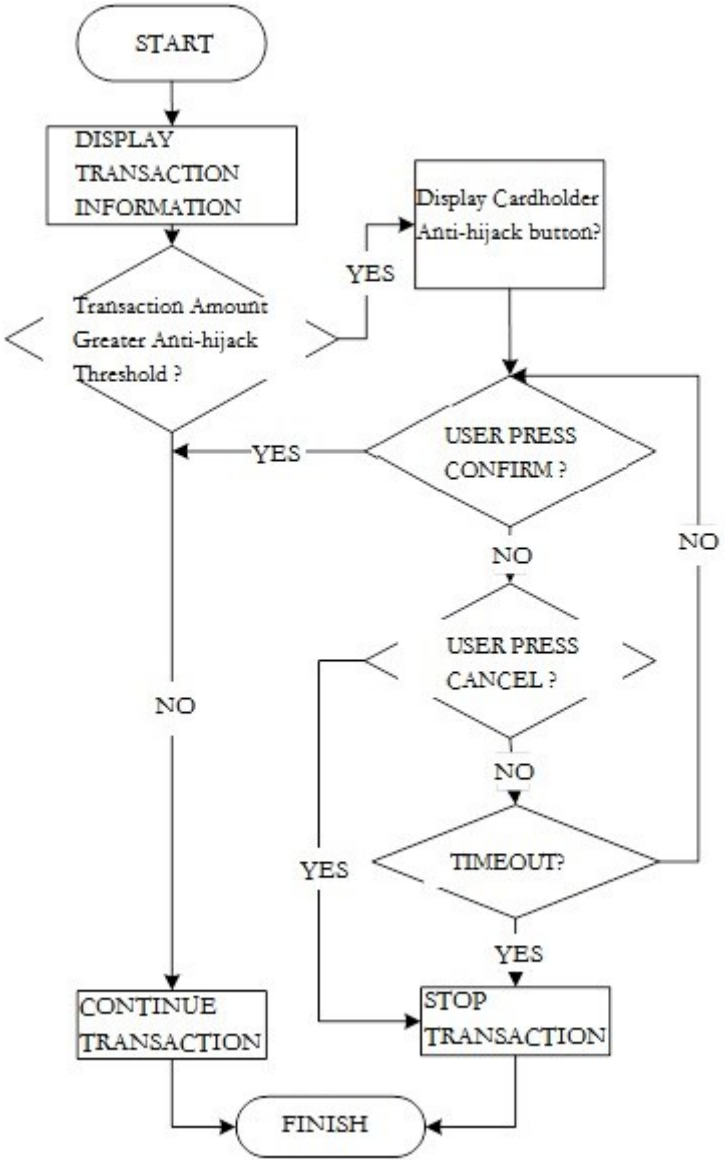


Figure 12 Transaction Anti-hijack Process

10.5 Online PIN Acquisition Mechanism

Online Pin acquisition refers to the online PIN acquisition process conducted by access devices; this process is recommended to be conducted by using the keyboard on the multi-interface card, or conducted by using the multi-interface card public key encryption method if keyboard is not present. See below for details:

- The cardholder uses the access device to input and confirm an online PIN;

- The access device sends a "INIT FOR PIN ENCRYPT (initialize PIN encryption)" command to the multi-interface card and acquires the multi-interface card's public key K_p and random number R (4 bytes); the multi-interface card will store R until the transaction is completed;
- The access device will pad the online PIN using the ANSI X9.8 method and then add a 1-byte PIN length to the front of the PIN and pad an F to 8 byte PINBLOCK to the PIN's right end (for example: if plaintext PIN is 123456, then PINBLOCK = = 0x06 0x12 0x34 0x56 0xFF 0xFF 0xFF 0xFF);
- The access device will connect random number R to the right end of PINBLOCK and then add 0 to the left end of PINBLOCK until the entire length of the data block is equal to the length of multi-interface card public key K_p; once this padding has been completed the data block will be tagged as D_1;
    - The access device will use K_p to conduct RSA encryption on data block D_1, thus creating encryption product X_1;
    - The access device will then send a "SET PIN BALANCE" command and PIN ciphertext X_1 to the multi-interface card;
    - The multi-interface card will then use its private key K_s to decrypt X_1 and obtain the plaintext of data block D_1;
    - The multi-interface card will extract random number R from data block D_1 and will then compare it to the random number stored on board; if they are the same the process will continue, if not the transaction will end;
    - The multi-interface card acquires the online PIN plaintext from data block D_1;
    - The multi-interface card will then assemble the online PIN plaintext and the other data described above into an online Message and then encrypt this Message using the dialogue key; this will then be sent to the processing center by way of the access device and thus complete the transaction.

10.6 Audio Access Mode Exception Handling

In audio access mode, due to some cellphone application (call, notification, media player etc) will also operate headset interface, communication interference may happen. Suggest to check audio communication API and other application status to limit interference. As different smart devices may use different operation systems and audio management methods, below anti-interference methods are not required.

1. Suggest Application detect call status when in transactions, and provide user alert, give user the choice to choose continue or pull out multi-interface card to answer call.
2. Suggest after insert multi-interface card, before transactions, change system 'notification' audio to mute, when transaction finish, resume with unmute.

3. If transaction is interrupted by answering call, notification audio or media player audio, etc, system should follow anti-pull out, anti-plugin mechanism to process.

10.7 Digital Certificate Security

Multi-interface card digital certificate functionality should support RSA, detail information refer to Appendix B.

11 Secure Channel Application

Secure channel applications are special applications that are used by the security modules of multi-interface cards and are used to establish secure channels.

Secure channel applications exist concurrently alongside UICS financial applications within multi-interface cards. UICS financial applications supply multi-interface cards with UICS transaction support; secure channel applications are responsible for guaranteeing the secrecy and integrity of online transaction data; offline transactions are not related to secure channel applications.

11.1 File Structure

The diagram below indicates the file structure of secure channel applications.

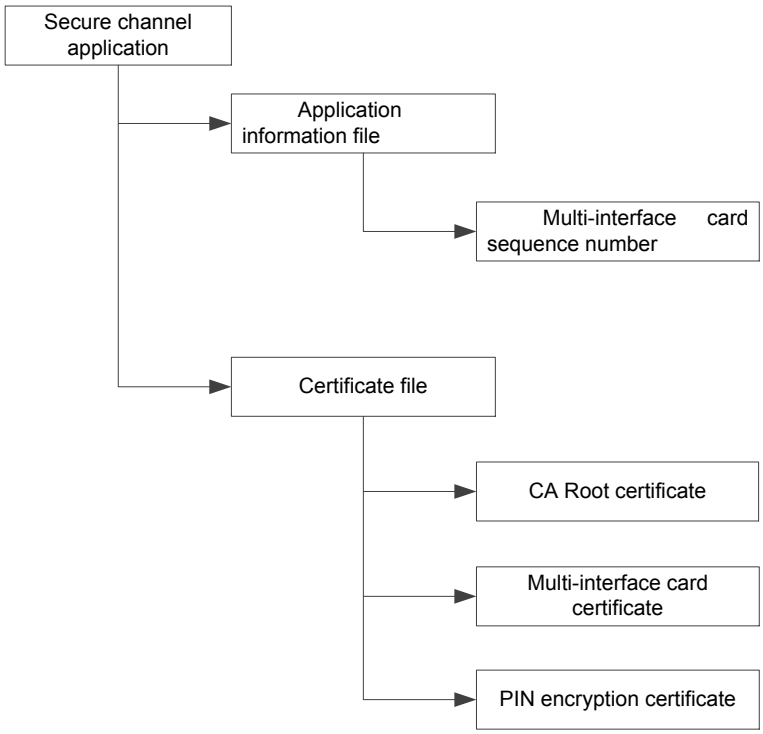


Figure 13 Secure Channel Application Structure File

- Application Information File

This file only stores multi-interface card sequence numbers; multi-interface card sequence numbers are composed of 23 digit with the following form:

Table 4 Multi-Interface Sequence Number Code Rules

Description	Length (in bytes)	Type
Affiliation Code	8	n

Description	Length (in bytes)	Type
Production Vendor Code	3	n
Production Date	4	n
Multi-Interface Card Identifier Code	8	an

- Certificate File

  This file is only used to store the certificates required for secure channel establishment processes, including CA Root certificates, multi-interface card certificates, and PIN encryption certificates. See 9.2 for detailed information regarding these certificates.

11.2 Application Instructions

11.2.1 READ DEVICE INFO Command

11.2.1.1 Definition and Scope

The READ DEVICE INFO command is used to acquire multi-interface card production vendor information, including firmware version numbers, multi-interface card status, and etc.

11.2.1.2 Command Message

See the table below for information on READ DEVICE INFO command Message code

Table 5 READ DEVICE INFO Command Message

Code	Value
CLA	7E
INS	10
P1	00
P2	00: Read multi-interface card device status /01: Read multi-interface card device information
Lc	Does not exist

Code	Value
Data	Does not exist
Le	See description below

11.2.1.3 Command Message Data Fields

Command Message data fields do not exist.

11.2.1.4 Response Message Data Fields

P2=0x00:

Indicates acquire multi-interface card status; returns 1-byte status information, Le=0x01. Status byte values indicate the following:

Table 6 Multi-Interface Card Status Byte Definitions

B7	B6	B5	B4	B3	B2	B1	B0	Note
							1	Multi-interface card sequence number already set
							0	Multi-interface card sequence number not yet set
						1		CA Root certificate already installed
						0		CA Root certificate not yet installed
					X			Retain
				1				PIN encryption certificate already installed
				0				PIN encryption certificate not yet installed
			1					Multi-interface card certificate already installed

			0					Multi-interface card certificate not yet installed
	0	0						Multi-interface card status: Power on
	1	0						Multi-interface card status: transaction approved (Channel certificate verification successful)
	X	X						Retain other values
X								Retain

P2=0x01:

Indicates acquire multi-interface card sequence number, firmware version number and multi-interface card number; the multi-interface card sequence number is the multi-interface card production number which is composed of the affiliation code (8 bytes) + production vendor code (3 bytes) + (production date (4 bytes in YYMM format) + multi-interface card identification number (8 bytes).

Output data is in TLV form. Response data is defined in the table below:

Table 7 READ DEVICE INFO Read Multi-Interface Information Response Message Data Fields

Description	Length (in bytes)	Note
Multi-Interface Card Production Number Data Label	1	Label value: 01
Multi-Interface Card Production Number Data Length	1	
Multi-Interface Card Production Number	23	
Multi-Interface Card Firmware Version Number Data Label	1	Label value: 02

Description	Length (in bytes)	Note
Multi-Interface Card Firmware Version Number Data Length	1	
Multi-Interface Card Firmware Version Number	1-16	
Public Key Version Number Data Label	1	Label value: 03
Public Key Version Number Data Length	1	
Public Key Version Number	2	00-99
IC Card Card Number Data Label	1	Label value: 04
IC Card Card Number Data Length	1	
IC Card Card Number Data	14-19	
Reversal Identifier Bit Data Label	1	Label value: 05
Reversal Identifier Bit Data Length	1	
Reversal Identifier Bit	1	0x30: No reversal information, 0x31: reversal information
Multi-Interface Card Type Number Data Label	1	Label value: 06
Multi-Interface Card Type Number Data Length	1	

Description	Length (in bytes)	Note
Multi-Interface Card Type Number	1-16	

11.2.1.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

11.2.2 EXCHANGE STATUS Command

11.2.2.1 Definition and Scope

EXCHANGE STATUS Command are used to process the status of processing centers and multi-interface cards.

11.2.2.2 Command Message

EXCHANGE STATUS command Message codes are described in the table below:

Table 8 EXCHANGE STATUS Command Messages

Code	Value
CLA	7E
INS	14
P1	Status (0-10 are used exclusively by the processing center and multi-interface cards)
P2	Reserve
Lc	Does not exist
Data	Does not exist
Le	See description

CLA='7E': Transparent transmission

CLA='7F': Encrypted transmission

Le='00': Indicates maximum byte number required (256 bytes)

11.2.2.3 Command Message Data Fields

Does not exist.

11.2.2.4 Response Message Data Fields

Does not exist.

11.2.2.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

The following table describes alarm status codes which may be returned by the terminal:

Table 9 EXCHANGE STATUS Alarm Statuses

SW1	SW2	Meaning
'6D'	'00'	INS not supported, or error
'6E'	'00'	CLA not supported, or error
'6A'	'86'	P1、P2 parameter incorrect
'69'	'85'	Usage requirements not satisfied

The following table describes error status codes which may be returned by the terminal:

Table 10 EXCHANGE STATUS Error Statuses

SW1	SW2	Meaning
'65'	'81'	Internal memory error
'67'	'00'	Length error
'62'	'83'	Selected file ineffective
'63'	'00'	Authentication failed
'69'	'00'	Unable to process

SW1	SW2	Meaning
‘69’	‘82’	Unsatisfactory security status
‘69’	‘83’	Verification method locked
‘69’	‘87’	Loss of security Message data item
‘6A’	‘81’	Function not supported
‘6F’	‘00’	Data ineffective
‘93’	‘03’	Application permanently locked
‘94’	‘03’	Key index not supported

Note: When P1 is 0-10, and 0x90XX is returned then XX is the transmission value of P1. For example: 0x7E 47 01 00 00; return: 0x9001

11.2.3 ADD CERTIFICATE Command

11.2.3.1 Definition and Scope

ADD CERTIFICATE commands are used to install new certificates for multi-interface cards.

11.2.3.2 Command Message

See the table below for information on ADD CERTIFICATE command Message codes:

Table 11 ADD CERTIFICATE Command Messages

Code	Value
CLA	7E
INS	20
P1	High 4bit indicates certificate category, low 4bit indicates offset value

Code	Value
P2	Offset value(low 8bit)
Lc	Certificate SubMessage Length
Data	Certificate SubMessage Data
Le	Does not exist

P1 high 4bit information defines an increase in certificate category; see table below for details:

Table 12 Certificate Category Identifier Defections

B7	B6	B5	B4	Explanation
0	0	0	1	Level 1 CA Root Certificate
0	0	1	0	Level 2 CA Root Certificate
0	1	0	0	PIN Encryption Certificate
1	0	0	0	Retain

P1 low 4bit and P2 byte is used to create a 12bit offset value; the scope of this offset value is 0~4095 bytes.

11.2.3.3 Command Message Data Fields

Command Message data field content includes certificate datagrams.

Limited by communication agreements, certificate data must be transmitted in segments; this means that ADD CERTIFICATE commands must be executed multiple times.

11.2.3.4 Response Message Data Fields

Response Message data fields do not exist.

11.2.3.5 Response Message Status Codes

“9000” designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

11.2.4 UPDATE CERTIFICATE Command

11.2.4.1 Definition and Scope

UPDATE CERTIFICATE commands are used to update certificates already held by multi-interface cards.

11.2.4.2 Command Messages

See the table below for information on UPDATE CERTIFICATE command Message codes:

Table 13 UPDATE CERTIFICATE Command Message

Code	Value
CLA	7E
INS	21
P1	High 4bit indicates certificate category, low 4bit indicates offset value
P2	Offset value(low 8bit)
Lc	Certificate Sub-message Length
Data	Certificate Sub-message Data
Le	Does not exist

P1 high 4bit information defines an increase in certificate grade; see table below for details:

Table 14 Certificate Category Identifier Defections

B7B7	B6	B5	B4	Explanation
0	0	0	1	Level 1 CA Root Certificate
0	0	1	0	Level 2 CA Root Certificate
0	1	0	0	PIN Encryption Certificate

B7B7	B6	B5	B4	Explanation
1	0	0	0	Retain

P1 low 4bit and P2 bytes are used to create a 12bit offset value; the scope of this offset value is 0~4095 bytes.

11.2.4.3 Command Message Data Fields

Command Message data field content includes updated certificate data.

Limited by communication agreements, certificate data must be transmitted in segments; this means that UPDATE CERTIFICATE commands must be executed multiple times.

11.2.4.4 Response Message Data Fields

Response Message data fields do not exist.

11.2.4.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

11.2.5 DELETE CERTIFICATE Command

11.2.5.1 Definition and Scope

DELETE CERTIFICATE commands are used to erase public key certificates already held by multi-interface cards.

11.2.5.2 Command Message

See the table below for information on DELETE CERTIFICATE command Message codes:

Table 15 DELETE CERTIFICATE Command Message

Code	Value
CLA	7E
INS	22
P1	Certificate Category Identifier (See Table 14)
P2	00

Code	Value
Lc	Data Field Data Length
Data	Multi-Interface Card Information
Le	Does not exist

P1 high 4bit information defines an increase in certificate grade; see table below for details:

Table 16 Certificate Category Identifier Definitions

B7	B6	B5	B4	Explanation
0	0	0	1	Level 1 CA Root Certificate
0	0	1	0	Level 2 CA Root Certificate
0	1	0	0	PIN Encryption Certificate
1	0	0	0	Retain

11.2.5.3 Command Message Data Fields

Multi-interface cards conduct verification of the information sent by the processing center; if verification results are the same, then the multi-interface card will erase the selected certificate category; otherwise this command execution will fail.

Table 17 DELETE CERTIFICATE Command Data Fields

Data	Length (in bytes)	Notes
Multi-Interface Card Sequence Number	23	
Firmware Version Number	1-16	

11.2.5.4 Response Message Data Fields

Response Message data fields do not exist.

11.2.5.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

11.2.6 READ CERTIFICATE Command

11.2.6.1 Definition and Scope

READ CERTIFICATE commands are used to read certificates already held by multi-interface cards.

11.2.6.2 Command Messages

See the table below for information on READ CERTIFICATE command Message codes:

Table 18 READ CERTIFICATE Command Message

Code	Value
CLA	7E
INS	23
P1	00: Read certificate; 01: Read certificate hash value
P2	Refer to table16 for detailed usage
Lc	Does not exist
Data	Does not exist
Le	See explanation

Table 19 Control Parameter Set Value Explanation

P2	Read Content	Note
0x00	Multi-Interface Card Certificate	

P2	Read Content	Note
0x01	Level 1 CA Root Certificate	
0x02	Level 1 CA Root Certificate	
0x03	PIN Encryption Certificate	

11.2.6.3 Command Message Data Fields

Command Message data fields do not exist.

11.2.6.4 Response Message Data Fields

Return to corresponding certificate data.

11.2.6.5 Response Message Status Codes

Status code "61FF" means that this command was executed successfully and indicates that a GET CERT RESPONSE command is required to read the response data. For possible error messages sent in response to this command, see Appendix C.

11.2.7 GET CERT RESPONSE Command

11.2.7.1 Definition and Scope

The GET CERT RESPONSE command is used to read the response data from the READ CERTIFICATE command.

11.2.7.2 Command Messages

See the table below for information on GET CERT RESPONSE command Message codes:

Table 20 GET CERT RESPONSE Command Message

Code	Value
CLA	7E
INS	24
P1	00

Code	Value
P2	00
Lc	Does not exist
Data	Does not exist
Le	Expected read certificate datagram length

11.2.7.3 Command Message Data Fields

Command Message data fields do not exist.

11.2.7.4 Response Message Data Fields

Certificate data response sent in accordance with set length

11.2.7.5 Response Message Status Codes

"9000" designates that this command was executed successfully; if the response code is '61XX', then this indicates that XX number of data remains to be read and that this command should be repeated so as to continue reading remaining certificate data.

For possible error messages sent in response to this command, see Appendix C.

11.2.8 GET CLIENT HELLO Command

11.2.8.1 Definition and Scope

GET CLIENT HELLO is used to acquire the algorithm identifier and random number supported by a multi-interface card.

11.2.8.2 Command Message

See the table below for information on GET CLIENT HELLO command Message codes:

Table 21 GET CLIENT HELLO Command Message

Code	Value
CLA	7E
INS	25

Code	Value
P1	00
P2	00
Lc	Does not exist
Data	Does not exist
Le	0x21

11.2.8.3 Command Message Data Fields

Command Message data fields do not exist.

11.2.8.4 Response Message Data Fields

Response data includes 0x01-byte algorithm identifiers and 0x20-byte random numbers. See the table below for algorithm descriptions.

Table 22 Algorithm Description Byte Definitions

B7	B6	B5	B4	B3	B2	B1	B0	Algorithm
*	*	*	*	*	*	*	1	RSA
*	*	*	*	*	*	1	*	ECC
*	*	*	1	*	*	*	*	3DES
*	*	1	*	*	*	*	*	Retain

11.2.8.5 Response Message Status Codes

“9000” designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

11.2.9 HASH SERVER CERTIFICATE Command

11.2.9.1 Definition and Scope

The HASH SERVER CERTIFICATE command is used to conduct summary operations on channel certificates; this summary result is compared against the decryption result of the VERIFY CERTIFICATE command.

11.2.9.2 Command Message

See the table below for information on HASH SERVER CERTIFICATE command Message codes:

Table 23 HASH SERVER CERTIFICATE Command Messages

Code	Value
CLA	7E
INS	26
P1	00
P2	00: SHA-1 Algorithm; Other: Retain
Lc	Input data length
Data	Input data
Le	Does not exist

11.2.9.3 Command Message Data Fields

The structure of the command data is: 1-byte mark bit + 1-byte extracted offset value + channel certificate subsegment data.

See the table below for 1-byte mark bit definitions:

Table 24 Meaning of Labeled Value Bytes

B7	Segment Certificate Information Starting Mark Bit
B6	Segment Certificate Information Ending Mark Bit

B5	Current Data Must Extract OU Field Mark Bit
B4	Current Data Must Extract Public Key Mark Bit
B3	Retain
B2	Retain
B1	Retain
B0	Retain

1-byte extract information offset byte: when an OU field or public key field must be extracted from a data subsegment, this indicates the byte length as measured from the first byte; this allows multi-interface card firmware to rapidly locate and extract the information.

11.2.9.4 Response Message Data Fields

Response Message data fields do not exist.

11.2.9.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

11.2.10 VERIFY SERVER CERTIFICATE Command

11.2.10.1 Definition and Scope

The VERIFY SERVER CERTIFICATE command is used to decrypt channel certificate signatures; the decryption result is then compared to the HASH SERVER CERTIFICATE summary value within the multi-interface card. If the values are the same then the channel certificate has been verified and the multi-interface card status will be changed to "transaction approved" (see Section 10.2.1.4 for multi-interface card status byte definitions).

11.2.10.2 Command Message

See the table below for information on VERIFY SERVER CERTIFICATE command Message codes:

Table 25 VERIFY SERVER CERTIFICATE Command Message

Code	Value
CLA	7E
INS	27
P1	00
P2	00: RSA Algorithm; Other: Retain
Lc	Input Data Length
Data	Input Data
Le	Does not exist

11.2.10.3 Command Message Data Fields

The structure of input command data is: 1-byte mark bit byte + 1-byte offset value + signature value subsegment data

See the table below for 1-byte mark bit definitions:

Table 26 Meaning of Mark Bit Byte Names

B7	Signature Value Starting Segment Mark Bit
B6	Signature Value Ending Segment Mark Bit
B5	Retain
B4	Retain
B3	Retain
B2	Retain

B1	Retain
B0	Retain

As accessed certificate signature values are in TLV code format, offset values indicate the offset distance of the value from the subsegment start address.

11.2.10.4 Response Message Data

Response Message data does not exist.

11.2.10.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

11.2.11 CLIENT SIGN Command

11.2.11.1 Definition and Scope

The CLIENT SIGN Command uses the multi-interface card's private key to sign designated data and return a signature value.

11.2.11.2 Command Message

See the table below for information on CLIENT SIGN command Message codes:

Table 27 CLIENT SIGN Command Message

Code	Value
CLA	7E
INS	28
P1	00
P2	00
Lc	Input Data Length
Data	Input Data
Le	00

11.2.11.3 Command Message Data Fields

Data to be signed.

11.2.11.4 Response Message Data Fields

Signed Data.

11.2.11.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

11.2.12 EXPORT MASTERKEY Command

11.2.12.1 Definition and Scope

The EXPORT MASTERKEY command is used to acquire shared main keys generated by multi-interface cards and then encrypt them using channel certificate public keys.

11.2.12.2 Command Messages

See the table below for information on EXPORT MASTERKEY command Message codes:

Table 28 EXPORT MASTERKEY Command Message

Code	Value
CLA	7E
INS	29
P1	00
P2	00
Lc	Does not exist
Data	Does not exist
Le	00

11.2.12.3 Command Message Data Fields

Command Message data does not exist.

11.2.12.4 Response Message Data Fields

Shared main key ciphertext information encrypted with channel certificate public key.

11.2.12.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

11.2.13 HMAC Command

11.2.13.1 Definition and Scope

The HMAC command is used to:

1) Acquires the HMAC value produced by the completion of the multi-interface handshake process and transmits it to the processing center for verification.
2) Inputs the HMAC value produced by the completion of the processing center handshake protocol process and transmits it to the multi-interface card for verification.
3) Generation of dialogue key by multi-interface card using HMAC algorithm.

11.2.13.2 Command Message

HMAC command Message codes are described in the table below:

Table 29 HMAC Command Message

Code	Value
CLA	7E
INS	2A
P1	00
P2	Control Parameter (See explanation)
Lc	Set value according to P2, see explanation below
Data	Set value according to P2, see explanation below
Le	Set value according to P2, see explanation below

Table 30 Control Parameter Set Value Table

P2	Lc	Data	Le	Note
0x00	Does not exist	Does not exist	0x14	HMAC value generated by completion of multi-interface card handshake
0x01	0x14	Processing Center HMAC Value	0x00	HMAC value generated by completion of processing center handshake
0x02	Does not exist	Does not exist	0x00	Dialogue key generated internally by HMAC algorithm.

11.2.13.3 Command Message Data Fields

See control parameter set value explanation.

11.2.13.4 Response Message Data Fields

When P2=00, response Message data fields are the HMAC values computed by the multi-interface card.

When P2=01 and P2=02, there are no response Message data fields.

11.2.13.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

11.2.14 TRANSMIT ENCRYPTED COMMAND Command

11.2.14.1 Definition and Scope

The TRANSMIT ENCRYPTE COMMAND command is for encrypted transmissions between the processing center and multi-interface cards.

11.2.14.2 Command Message

See the following table for TRANSMIT ENCRYPTED COMMAND command Message codes:

Table 31 TRANSMIT ENCRYPTED COMMAND Command Message

Code	Value
CLA	7F
INS	2B

Code	Value
P1	00
P2	00:Non-Cascade Method; 01: Cascade Method
Lc	Input Data Length
Data	Input Data
Le	00

When P2=0x00, secure channel information Message are being transmitted using a non-cascade method or the information Message has already reached the final segment of the data cascade.

When P2=0x01, secure channel information Message are being transmitted in a cascade and there is more data to follow.

11.2.14.3 Command Message Data Fields

SKey-encrypted command data sent by the processing center and MKey-calculated MAC.

11.2.14.4 Response Message Data Fields

Encrypted command response data and MAC.

11.2.14.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

Note:

(1) Transaction commands can only be transmitted to multi-interface cards using this encryption command; that is to say that multi-interface card transactions can only be conducted after a secure channel has been established.

(2) Multi-interface card commands are transmitted using this command after decryption; execution results and return codes must be encrypted before they are sent to the processing center, and return codes must be sent directly after return ciphertexts in plaintext format.

Example: the '【】' (brackets) below indicate APDU command data fields; this field includes two parts; the encrypted plaintext data includes within the brackets as well as MAC ciphertext.

Processing center command: 7F 2B 00 00 10 【00 05 7E 2C 00 00 00】

Multi-interface card return data: 【90 00】90 00 90 00

11.2.15 CLOSE SECURE CHANNEL Command

11.2.15.1 Definition and Scope

CLOSE SECURE CHANNEL commands are used to close secure channels and destroy all keys used within that channel.

11.2.15.2 Command Message

See the table below for information on CLOSE SECURE CHANNEL command Message codes:

Table 32 CLOSE SECURE CHANNEL Command Message

Code	Value
CLA	7E
INS	2C
P1	00
P2	00
Lc	Does not exist
Data	Does not exist
Le	00

11.2.15.3 Command Message Data Fields

Command Message data fields do not exist.

11.2.15.4 Response Message Data Fields

Response Message data fields do not exist.

11.2.15.5 Response Message Status Codes

“9000” designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

11.2.16 INIT FOR PIN ENCRYPT Command

11.2.16.1 Definition and Scope

INIT FOR PIN ENCRYPT (initialize PIN encryption) commands are used to acquire PIN transmission protection keys; namely multi-interface card public keys. This is to guarantee that the PIN is in ciphertext form when transmitted from the access device to the card.

11.2.16.2 Command Message

See the following table for INIT FOR PIN ENCRYPT command Message codes:

Table 33 INIT FOR PIN ENCRYPT Command Message

Code	Value
CLA	7E
INS	55
P1	00
P2	00
Lc	Does not exist
Data	Does not exist
Le	00

11.2.16.3 Command Message Data Fields

Command Message data does not exist.

11.2.16.4 Response Message Data Fields

Response Message data returns multi-interface card public key, 4-byte random number; returned data is in TLV format.

11.2.16.5 Response Message Status Codes

“9000” designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

11.2.17 SET PIN BALANCE Command

11.2.17.1 Definition and Scope

The SET PIN BALANCE command is used to transmit online PIN or balances from the access device to the multi-interface card.

11.2.17.2 Command Message

See the table below for information on SET PIN BALANCE command Message codes:

Table 34 SET PIN BALANCE Command Message

Code	Value
CLA	7E
INS	52
P1	01: Send Online PIN; 02: Send Balance
P2	00
Lc	Input Data Length
Data	Input Data
Le	00

11.2.17.3 Command Message Data Fields

P1＝01: The data field is the PIN encrypted by the multi-interface card public key.

P1＝02: Data field is balance plaintext.

11.2.17.4 Response Message Data Fields

Response Message data fields do not exist.

11.2.17.5 Response Message Status Codes

“9000” designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

Appendix A Secure Channel Establishment Process Example

The access device sends a GET CLIENT HELLO command to the multi-interface card. The card then responds with a 1-byte algorithm identifier and 32-byte random number;

The access device uses the 1-byte algorithm identifier sent by the multi-interface card to acquire the list of symmetrical and asymetrical algorithms supported by the card; this identifier is transmitted to the server, which uses the list to determine if the signature algorithm and symmetrical algorithm used by the server's channel certificate is supported;

The random number is used after this point for verification purposes and generation of authentication information.

The server then sends the channel certificate, algorithm identifier, and 32-byte random number to the multi-interface card.

At this point, the multi-interface card should first inspect the legality of the channel certificate using HASH SERVER CERTIFICATE and VERIFY SERVER CERTIFICATE commands;

The HASH SERVER CERTIFICATE command is used to conduct a hash operation on the main body of the channel certificate and then save the result of the hash operation within the multi-interface card;

The VERIFY SERVER CERTIFICATE command decrypts the CA Root channel certificate signature value within the multi-interface card and then compares it to the hash result of the HASH SERVER CERTIFICATE command; this determines the legality of the channel certificate;

Once the legality of the channel certificate has been determined the multi-interface card will generate a 48-byte random number as a shared main key and then encrypt this key using the channel certificate; the EXPORT MASTERKEY command will then complete the encryption of the shared main key by the channel certificate and return a 128-byte ciphertext;

The multi-interface card certificate will then be read; using the READ CERTIFICATE and GET CERT RESPONSE commands; because certificates are larger than the maximum transmission bytes admissible by CCID, making it impossible to read a multi-interface card certificate using a single command. The GET CERT RESPONSE command can be used multiple times until the entire multi-interface card certificate has been read; for command details, see 10.2;

When the server authenticates the multi-interface card it must do so by checking the card’s private key signature; the card will use the CLIENT SIGN command to conduct hash and signature operations on the connected value of the access server’s random number and the card’s own random number, and then return 128-byte signature data;

The multi-interface card then sends the 128-byte shared main key ciphertext, multi-interface card certificate and 128-byte signature data to the server.

The server uses the root certificate to verify the legality of the multi-interface card certificate; if successful then it will verify the signature value with the multi-interface card public key and thus verify the legality of the card itself; once the card has been authenticated the channel certificate private key will be used to decrypt the shared main key ciphertext and acquire the 48-byte shared main key; here, a server authentication completion message must be sent; so as to guard against this message being fraudulent it must be completed using an HMAC calculation; the first 16 values of the 48-byte shared main key serves as the key with the data consisting of ASCII"SERVER", the multi-interface card random number, the server random number, the channel certificate hash value, the multi-interface card hash value, the signature value sent from multi-interface card to the server, and the shared main key ciphertext;

The server will then take the handshake completion message (the server-calculated HMAC value) and send it to the multi-interface card.

Once the multi-interface card has received the HMAC it will use the HMAC (P2=0x01) command to verify the HMAC value produced by the server during handshake completion. Then the HMAC (P2=0x00) command will be used to return the HMAC value to the multi-interface card; this process is the same as used by the server to generate an HMAC value, but changes ASCII"SERVER" to "CLIENT";

The server will then take the handshake completion message (the multi-interface card-calculated HMAC value) and send it to the server;

The multi-interface card will then use the HMAC (P2=0x02) command to generate a dialogue key; this dialogue key will only be stored within the multi-interface card and will not be exported; in the case of a power failure this key must be regenerated.

After the server has verified the multi-interface card handshake completion message the dialogue key will be generated.

Finally, both the server and multi-interface card will have a 48-byte shared main key and a 20-byte dialogue key; the first 16 bytes of the 20-byte dialogue key acts as the encryption key while the last 16-bytes act as the key used to calculate MAC.

Appendix B Algorithms

B.1 Data Encryption

The data encryption process used for updating keys is as follows:

Step 1

Data block generation:

- Plaintext data length (Ld)
- Plaintext data (Ld placed before plaintext)

Step 2

The data block generated in step one is divided into 8-byte units with the labels D1, D2, D3, D4, etc; the length of last data block may between 1 and 8 bytes.

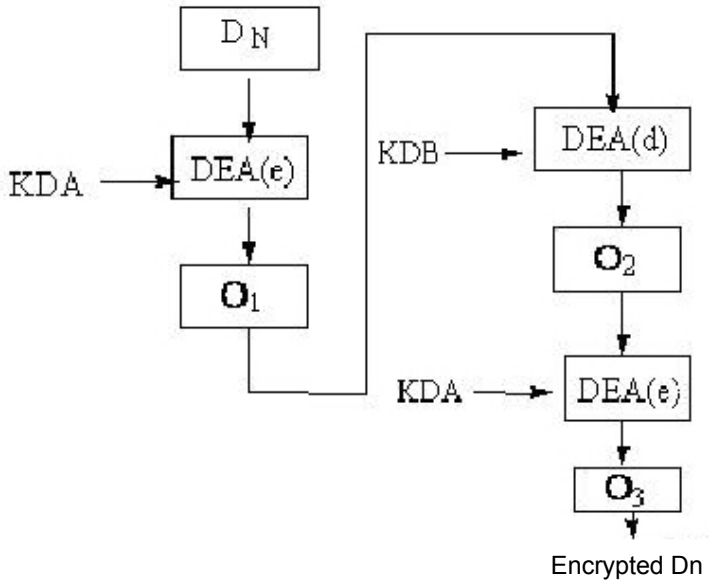
Step 3

Pad the final data block using the method indicated below until it reaches 8bytes:

- If the data block is already 8-bytes then no additional data needs to be added.
- If the last data block does not yet have 8 bytes then add a hexadecimal '80'; if this is not enough to reach 8 bytes then add the hexadecimal '00' until it does.

Step 4

Use a 3DES calculation to encrypt the data block; see diagram below.



O=Output DEA(e)=DES encryption calculation

D=data block DEA(d)=DES decryption calculation

KDA=encryption protection key (DAMK) left 8-bytes

KDB=encryption protection key (DAMK) right 8-bytes

Step 5

All encrypted data blocks should be connected to one another according to their original sequence (post-encryption D1, D2, etc.) to create the final ciphertext data. Most ciphertext data is instructional data fields sent to the card.

B.2 Grouping Algorithm-Based MAC

MAC calculations are in accordance with ISO/IEC 9797-1 specifications and uses 128-digit length key 3DES encryption calculations in CBC mode to derive 8-byte MAC values from a message of any length.

- Algorithm Parameters

M	Plaintext Message
C	Ciphertext Message
M_{AC}	Message Authentication Code
K	MAC Key
IV	Initial Vector
$E_K(M)$	M encrypted with Key K
$D_K(C)$	C encrypted with Key K

- Algorithm Process

1. Padding Group

Add 0x80 to the end of plaintext M and then as many 0x00 to the right end as required such that M = (M||80||00||00||…||00) is a whole multiple of 8. Group M into the 16-byte segments $M_1, M_2,\ldots,M_n$.

2. Calculating MAC

The 3DES algorithm using CBC mode is used to encrypt left half 8-byte KL groups $M_1, M_2,\ldots,M_n$ of key K. The initial vector IV = (00 || 00 || 00 || 00 || 00 || 00 || 00 || 00).

The CBC mode encryption process is as follows:

$$C_0 = IV$$

$$C_i = E_{KL}(M_i \oplus C_{i-1}) , i = 1, 2, \ldots , n$$

The method used to calculate the message authentication code from the last data block is the following:

$M_{AC} = E_{KL}(D_{KR}(C_n))$

B.3 Hash Calculation–Based HMAC

- Algorithm Parameters

HMAC calculations are in accordance with FIPS specifications and use the SHA-1 algorithm to generate a 20-byte HMAC.

Table B.1 HMAC Algorithm Parameter Explanation

ipad	Pad byte string with content: 8-byte 0x36, repeat 64 times
opad	Pad byte string with content: 8-byte 0x5c, repeat 64 times
text	All data requiring MAC calculation for input, not including pad byte strings
K	MAC Key
t	All MAC byte lengths
SHA-1 Secure Hash Algorithm	M encrypted with Key K
$D_K(C)$	See FIPS 180-3

- Algorithm Process

The MAC value of input text is generated using the following equation:

$MAC(text)_t = HMAC(K, text)_t = SHA\text{-}1\ ((K_0 \oplus opad) \| SHA\text{-}1((K_0 \oplus ipad) \| text))$

Specific explanation is as follows:

1. If K = 64, then make K_0 = K. Skip to step 4;
2. If K > 64, then make K_0 = SHA-1(K). Skip to step 4;
3. If K < 64, then pad the end of K with 0x00 to create the 64-byte K_0;
4. K_0 and ipad XOR generates the 64-byte byte string: $K_0 \oplus$ ipad;
5. Add text to the end of the byte string $K_0 \oplus$ ipad generated in step 4: ($K_0 \oplus$ ipad)||text;

6. Acquire hash of byte string generated in step 5: SHA-1(($K_0 \oplus$ ipad)||text);
7. $K_0$and opad XOR: $K_0 \oplus$ opad;
8. Add the result of step 6 to the end of the result of step 7:

$$(K_0 \oplus \text{opad}) \,||\, \text{SHA-1}((K_0 \oplus \text{ipad}) \,||\, \text{text})$$

9. Derive the hash value of the result of step 8:

$$\text{SHA-1}((K_0 \oplus \text{opad}) \,||\, \text{SHA-1}((K_0 \oplus \text{ipad}) \,||\, \text{text})).$$

10. The 20 byte hash value derived from step 9 is the MAC value.

B.4 RSA Encryption Algorithm

The RSA public key is used to encrypt the plaintext message. The encryption standard is the PKCS#1 specification encryption modulus RSAES-PKCS1-v1_5.

- Algorithm Parameters

M	Plaintext message
mLen	Length of M
EM	Post-editing plaintext message
C	Ciphertext message
K	RSA Public key modulus length

- Algorithm Process

1. Message Editing:
   a) Generate a non-zero random number byte string PS with a length of k - mLen – 3; PS must be at least 8-bytes in length.
   b) Connect PS to M using the method shown below, creating edited message EM with k-bytes.

$$EM = 0x00 \,||\, 0x02 \,||\, PS \,||\, 0x00 \,||\, M$$

Encryption: Use RSA public key to encrypt EM and generate ciphertext C.

B.5 RSA Signature Algorithm

RSA private key is used to sign message summary: the signature standard uses RSASSA-PKCS1-v1_5 of PKCS # 1 specifications. Message summary uses SHA-1 algorithm.

- Algorithm Parameters

M	Plaintext message
EM	Post-editing plaintext message
C	Ciphertext message
K	RSA public key modulus length

- Algorithm Process

1. Calculate message summary H from message M; the Hash algorithm used should be SHA-1:

$$H = SHA\text{-}1(M)$$

2. Message Editing:

a) Generate DER edit T of message H with length tLen:

T = (0x) 30 21 30 09 06 05 2b 0e 03 02 1a 05 00 04 14 || H

b) Generate byte string PS with consisting of k－tLen－3 hexadecimal 0xFF groups.

c) Connect PS to T using the method shown below, creating edited message EM with k-bytes.

$$EM = 0x00‖0x01‖PS‖0x00‖T$$

3. Signature:

The RSA private key is used to sign EM.

Appendix C Command Response Status Codes

SW1	SW2	Meaning
'90'	'00'	Normal processing
'61'	'XX'	Normal processing, 'XX' indicates that additional data lengths can be acquired with subsequent commands
'6C'	'XX'	Length error (Le is incorrect, 'XX' indicates actual length)
'65'	'81'	Multi-interface card device error
'69'	'82'	Security status insufficient
'69'	'85'	Usage requirements insufficient
'69'	'86'	Command is not allowed
'69'	'88'	Secure message data object is inaccurate
'67'	'00'	Data length error
'6A'	'80'	Data object does not exist
'6A'	'84'	Multi-interface card device does not have sufficient memory
'6A'	'86'	P1、P2 parameters inaccurate
'6D'	'00'	INS not supported or in error
'6E'	'00'	CLA not supported or in error

Appendix D Transaction Command List

D.1 CREDIT FOR LOAD Command

D.1.1 Definition and Scope

The CREDIT FOR LOAD command is used to support financial IC card online load transactions; it allows financial IC card main account funds to be marked as electronic cash funds and completes IC card electronic cash remainder update operations. Load amount input is completed using the virtual keyboard of the access device.

D.1.2 Command Messages

See the table below for CREDIT FOR LOAD command Message codes:

Table D.1 CREDIT FOR LOAD Command Messages

Code	Value
CLA	7E
INS	40
P1	00/01
P2	When P1=00 and P2=00: Start transaction, read data as defined in Table A.36; When P1=00 and P2=01: read data as defined in Table A.37; When P1=01 and P2=00: Online return of data
Lc	Data field data length
Data	See explanation of command Message data fields
Le	00

D.1.3 Command Message Data Fields

When P1=00 and P2=00:

Start load transaction; see the command Message data field table below:

Table D.2 Start Load Transaction Command Message Data Fields

Data	Length (in bytes)	Remarks
Transaction Amount	6	Set as 0
Transaction Date YYMMDD	3	Current date at processing center
Transaction Time HHMMSS	3	Current time at processing center
Other Transaction Data	Variable(VAR)	See notes below table

Notes: Other transaction date refers to processing center data; during abnormal transactions (such as reversals, script notifications, and etc.) this data must be sent back to the processing center in abnormal (such as reversals and script notifications) transaction Messages. Multi-interface cards do not need to analyze this data. Other transaction processing methods are the same.

When P1=00 and P2=01:

The command Message does not contain data fields.

When P1=01:

Online returned data; see the table below for command Message data fields:

Table D.3 Load Online Data Command Message Data Fields

Data	Length (in bytes)	Remarks
Online Result	1	00: Normal connection; 01: No connection
Issuer Authorization Data (Label 91)	10-18	TLV format
Authorization Response Code (Label 8A)	4	TLV format
71 Script Data (Label 71)	Variable	TLV format
72 Script Data (Label 72)	Variable	TLV format

D.1.4 Response Message Data Fields

When P1=00 and P2=00:

Start load transaction; see the response Message data field table below:

Table D.4 Start Load Transaction Response Message Data Fields

Data	Length (in bytes)	Remarks
Load Amount	6	Actual load amount
Transaction Result	1	01: Transaction refused; 02: Request to connect
Terminal Verification Result (Label 95)	7	TLV format
Transaction Date (Label 9A)	5	TLV format
Random Number (Label 9F37)	7	TLV format
Application Interchange Profile(AIP) (Label 82)	4	TLV format
Application Transaction Code (ATC)(Label 9F36)	5	TLV format
Ciphertext Information Type (CID) (Label 9F27)	4	TLV format
Application Ciphertext (AC) (Label 9F26)	11	TLV format
Issuer Application Data (Label 9F10)	Maximum 35	TLV format
Cardholder Verification Method (CVM) Result (Label 9F34)	6	TLV format

Data	Length (in bytes)	Remarks
Transaction Sequence Counter (Label 9F41)	5-7	TLV format
Specialized File Name (Label 84)	7-18	TLV format
Application Version Number (Label 9F09)	5	TLV format
Authorized Amount (Label 9F02)	9	Actual load amount

When P1=00 and P2=01:

Read PAN, PAN sequence number, and related information: see the command response data field table below:

Table D.5 Response Message Data Fields

Data	Length (in bytes)	Remarks
PAN (Label 5A)	Maximum 12	Application account number, TLV format
PAN Sequence Number (Label 5F34)	4	Application account number sequence number, TLV format
Magnetic Strip-Equivalent Data (Label 57)	Maximum 21	Magnetic strip-equivalent data, TLV format
Online PIN Ciphertext (Label 99)	130	PIN/public key encrypted-online PIN ciphertext

When P1=01:

Returned online data sent from multi-interface card to processing center; see the response Message data field table below:

Table D.6 Load Online Response Message Data Fields

Data	Length (in bytes)	Remarks
Transaction Result	1	00: Transaction approved; 01: Transaction denied
Script Execution Result (Label DF31)	8	TLV format
Terminal Verification Result (TVR) (Label 95)	7	TLV format
Application Transaction Code (ATC) (Label 9F36)	5	TLV format
Ciphertext Information Type(CID) (Label 9F27)	4	TLV format
Application Ciphertext (AC) (Label 9F26)	11	TLV format
Issuer Application Data (Label 9F10)	Maximum 35	TLV format

D.1.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

D.2 DEBIT FOR PURCHASE Command

D.2.1 Definition and Scope

The DEBIT FOR PURCHASE command supports only consumer transactions for financial IC cards; it allows the cardholder to use a financial IC card to complete internet purchases and acquire related services.

D.2.2 Command Message

See the table below for information on DEBIT FOR PURCHASE command Message codes:

Table D.7 DEBIT FOR PURCHASE Command Message

Code	Value
CLA	7E
INS	41
P1	00/01
P2	When P1=00 and P2=00: Start transaction, read data as defined in Table A.42; When P1=00 and P2=01: read data as defined in Table A.43; When P1=01 and P2=00: Online data return
Lc	Data field data length
Data	See explanation of command Message data fields
Le	00

D.2.3 Command Message Data Fields

When P1=00 and P2=00:

Start online consumption transaction; see the command Message data field table below:

Table D.8 Command Message Data Fields

Data	Length (in bytes)	Remarks
Transaction Amount	6	Consumption amount
Transaction Date YYMMDD	3	Current date at processing center
Transaction Time HHMMSS	3	Current time at processing center
Other Transaction Data	Variable(VAR)	See notes below table

Notes: Other transaction date refers to processing center data; during abnormal transactions (such as reversals, script notifications, etc.) this data must be sent back to the processing center in abnormal (such as reversals and script notifications) transaction Messages. Multi-interface cards do not need to analyze this data. Other transaction processing methods are the same.

When P1=00 and P2=01:

The command Message does not contain data fields.

When P1=01:

Online returned data; see the table below for command Message data fields:

Table D.9 Command Message Data Fields

Data	Length (in bytes)	Remarks
Connection Result	1	00: Normal connection; 01: No connection
Issuer Authorization Data (Label 91)	10-18	TLV format
Authorization Response Code (Label 8A)	4	TLV format
71 Script Data (Label 71)	Variable	TLV format
72 Script Data (Label 72)	Variable	TLV format

D.2.4 Response Message Data Fields

When P1=00 and P2=00:

Start online consumption transaction; see the command Message data field table below:

Table D.10 Response Message Data Fields

Data	Length (in bytes)	Remarks
Transaction result	1	01: Transaction refused; 02: Connection request

Data	Length (in bytes)	Remarks
Terminal Verification Result (TVR) (Label 95)	7	TLV format
Transaction Date (Label 9A)	5	TLV format
Random Number (Label 9F37)	7	TLV format
Application Interchange Profile (AIP) (Label 82)	4	TLV format
Application Transaction Code (ATC) Label 9F36)	5	TLV format
Ciphertext Information Type (CID) (Label 9F27)	4	TLV format
Application Ciphertext (AC) (Label 9F26)	11	TLV format
Issuer Application Data (Label 9F10)	Maximum 35	TLV format
Cardholder Verification Method (CVM) Result (Label 9F34)	6	TLV format
Transaction Sequence Counter (Label 9F41)	5-7	TLV format
Specialized Document Name (Label 84)	7-18	TLV format
Application Version Number (Label 9F09)	5	TLV format

When P1=00 and P2=01:

Read PAN, PAN sequence number, and related information: see the command response data field table below:

Table D.11 Response Message Data Fields

Data	Length (in bytes)	Remarks
PAN (Label 5A)	Maximum 12	Application account number, TLV format
PAN Sequence Number (Label 5F34)	4	Application account number sequence number, TLV format
Magnetic Strip-Equivalent Data (Label 57)	Maximum 21	Magnetic strip-equivalent data, TLV format
Online PIN Ciphertext (Label 99)	130	PIN/public key encrypted-online PIN ciphertext

When P1=01:

Returned online data sent from multi-interface card to processing center; see the response Message data field table below:

Table D.12 Response Message Data Fields

Data	Length (in bytes)	Remarks
Transaction result	1	00: Transaction approved; 01: Transaction refused
Script Execution Result (Label DF31)	8	TLV format
Terminal Verification Result (TVR) (Label 95)	7	TLV format
Application Transaction Code (ATC) (Label 9F36)	5	TLV format
Ciphertext Information Type (CID) (Label 9F27)	4	TLV format

Data	Length (in bytes)	Remarks
Application Ciphertext (AC) (Label 9F26)	11	TLV format
Issuer Application Data (Label 9F10)	Maximum 35	TLV format

D.2.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

D.3 GET ELECTRONIC CASH BALANCE Command

D.3.1 Definition and Scope

The GET ELECTRONIC CASH BALANCE command is used for inspecting the electronic cash balance of a multi-interface card; this transaction occurs offline.

D.3.2 Command Messages

See the table below for information on GET ELECTRONIC CASH BALANCE command Message codes:

Table D.13 GET ELECTRONIC CASH BALANCE Command Message

Code	Value
CLACLA	7E
INS	42
P1	00: Displays electronic cash balance
P2	00
Lc	Does not exist
Data	Does not exist
Le	00

D.3.3 Command Message Data Fields

Command Message data fields do not exist.

D.3.4 Response Message Data Fields

See table below for financial IC card electronic cash balance/response Message data fields:

Table D.14 GET ELECTRONIC CASH BALANCE Response Message Data Fields

Explanation	Length (in bytes)	Remarks
Electronic Cash Balance	6	BCD Code

D.3.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

D.4 GET PRIMARY BALANCE Command

D.4.1 Definition and Scope

The GET PRIMARY BALANCE command is used for online inspection of debit/credit application main account balances.

D.4.2 Command Message

See the table below for information on GET PRIMARY BALANCE command Message codes:

Table D.15 GET PRIMARY BALANCE Command Message

Code	Value
CLA	7E
INS	43
P1	00/01
P2	When P1=00 and P2=00: Start transaction, read data as defined in Table A.50; When P1=00 and P2=01: read data as defined in Table A.51; When P1=01 and P2=00: Online data return

Code	Value
LcLc	Data field data length
Data	See explanation of command Message data fields
Le	00

D.4.3 Command Message Data Fields

When P1=00 and P2=00:

Begins debit/credit main account balance inspection transaction; see command Message data field table below:

Table D.16 Command Message Data Fields

Data	Length (in bytes)	Remarks
Transaction Amount	6	Balance set as 0
Transaction Date YYMMDD	3	Current date at processing center
Transaction Time HHMMSS	3	Current time at processing center
Other Transaction Data	Variable (VAR)	See notes below table

Notes: Other transaction data refers to processing center data; during abnormal transactions (such as reversals, script notifications, etc.) this data must be sent back to the processing center . Multi-interface cards do not need to analyze this data.

When P1=00 and P2=01:

The command Message does not contain data fields.

When P1=01:

Online returned data; see the table below for command Message data fields:

Table D.17 Command Message Data Fields

Data	Length (in bytes)	Remarks
Connection Result	1	00: Normal connection; 01: Unable to connect
Main Account Balance	6	BCD
Issuer Authorization Data (Label 9F)	10-18	TLV format
Authorization Response Code (Label 8A)	4	TLV format
71 Script Data (Label 71)	Variable	TLV format
72 Script Data (Label 72)	Variable	TLV format

D.4.4 Response Message Data Fields

When P1=00 and P2=00:

Begins debit/credit master account balance inquiry transaction; see command Message data field table below:

Table D.18 Response Message Data Fields

Data	Length (in bytes)	Remarks
Transaction result	1	01: Transaction refused; 02: Connection request
Terminal Verification Result (TVR) (Label 95)	7	TLV format
Transaction Date (Label 9A)	5	TLV format
Random Number (Label 9F37)	7	TLV format

Data	Length (in bytes)	Remarks
Application Interchange Profile (AIP) (Label 82)	4	TLV format
Application Transaction Code (ATC) Label 9F36)	5	TLV format
Ciphertext Information Type (CID) (Label 9F27)	4	TLV format
Application Ciphertext (AC) (Label 9F26)	11	TLV format
Issuer Application Data (Label 9F10)	Maximum 35	TLV format
Cardholder Verification Method (CVM) Result (Label 9F34)	6	TLV format
Transaction Sequence Counter (Label 9F41)	5-7	TLV format
Specialized Document Name (Label 84)	7-18	TLV format
Application Version Number (Label 9F09)	5	TLV format

When P1=00 and P2=01:

Read PAN, PAN sequence number, and related information: see the command response data field table below:

Table D.19 Response Message Data Fields

Data	Length (in bytes)	Remarks
PAN (Label 5A)	Maximum 12	Application main account number

Data	Length (in bytes)	Remarks
PAN Sequence Number (Label 5F34)	4	Application main account sequence number
Magnetic Strip-Equivalent Data (Label 57)	Maximum 21	Magnetic strip-equivalent data
Online PIN Ciphertext (Label 99)	130	PIN/public key encrypted-online PIN ciphertext

When P1=01:

Returned online data sent from multi-interface card to processing center; see the response Message data field table below:

Table D.20 Response Message Data Fields

Data	Length (in bytes)	Remarks
Transaction result	1	00: Transaction approved; 01: Transaction refused
Script Execution Result (Label DF31)	8	TLV format
Terminal Verification Result (TVR) (Label 95)	7	TLV format
Application Transaction Code (ATC) (Label 9F36)	5	TLV format
Ciphertext Information Type (CID) (Label 9F27)	4	TLV format
Application Ciphertext (AC) (Label 9F26)	11	TLV format

Data	Length (in bytes)	Remarks
Issuer Application Data (Label 9F10)	Maximum 35	TLV format

D.4.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

D.5 CREDIT CARD PAYMENT Command

D.5.1 Definition and Scope

Credit card repayment transaction command.

D.5.2 Command Message

See the table below for CREDIT CARD PAYMENT command Message codes:

Table D.21 CREDIT CARD PAYMENT Command Message

Code	Value
CLA	7E
INS	49
P1	00: Start transaction 01: Online return of data
P2	P2=00: Read Part 55 content, P2=01: Read online PIN ciphertext, PAN, and other non-55 field data
Lc	Input data length changes according to changes in P1 parameters
Data	See data field explanation below
Le	'00

D.5.3 Command Message Data Fields

When P1=00 and P2=00:

Start credit repayment transaction; see the command Message data field table below:

Table D.22 Command Message Data Fields

Data	Length (in bytes)	Remarks
Transaction Amount	6	Repayment amount
Transaction Date YYMMDD	3	Current date at processing center
Transaction Time HHMMSS	3	Current time at processing center
Credit Card Account Number	28	Pad from right with spaces if not 28
Other Transaction Data	Variable (VAR)	See notes below table

Notes: Other transaction date refers to processing center data; during abnormal transactions (such as reversals, script notifications, etc.) this data must be sent back to the processing center in abnormal (such as reversals and script notifications) transaction messages. Terminals do not need to analyze this data.

When P1=00 and P2=01:

Command data fields do not exist.

When P1=01:

Online returned data; see the table below for command Message data fields:

Table D.23 Command Message Data Fields

Data	Length (in bytes)	Remarks
Connection Result	1	00: Normal connection; 01: Unable to connect
Issuer Authorization Data (Label 9F)	10-18	TLV format

Data	Length (in bytes)	Remarks
Authorization Response Code (Label 8A)	4	TLV format
71 Script Data (Label 71)	Variable	TLV format
72 Script Data (Label 72)	Variable	TLV format

D.5.4 Response Message Data Fields

When P1=00 and P2=00:

Start credit card repayment transaction; see the command Message data field table below:

Table D.24 Response Message Data Fields

Data	Length (in bytes)	Remarks
Transaction result	1	01: Transaction refused; 02: Connection request
Terminal Verification Result (TVR) (Label 95)	7	TLV format
Transaction Date (Label 9A)	5	TLV format
Random Number (Label 9F37)	7	TLV format
Application Interchange Profile (AIP) (Label 82)	4	TLV format
Application Transaction Code (ATC) Label 9F36)	5	TLV format
Ciphertext Information Type (CID) (Label 9F27)	4	TLV format

Data	Length (in bytes)	Remarks
Application Ciphertext (AC) (Label 9F26)	11	TLV format
Issuer Application Data (Label 9F10)	Maximum 35	TLV format
Cardholder Verification Method (CVM) Result (Label 9F34)	6	TLV format
Transaction Sequence Counter (Label 9F41)	5-7	TLV format
Specialized Document Name (Label 84)	7-18	TLV format
Application Version Number (Label 9F09)	5	TLV format

When P1=00 and P2=01:

Read PAN, PAN sequence number, and related information: see the command response data field table below:

Table D.25 Response Message Data Fields

Data	Length (in bytes)	Remarks
PAN (Label 5A)	Maximum 12	Application account number, TLV format
PAN Sequence Number (Label 5F34)	4	Application account number sequence number, TLV format
Magnetic Strip-Equivalent Data (Label 57)	Maximum 21	Magnetic strip-equivalent data, TLV format
Online PIN Ciphertext (Label 99)	130	PIN/public key encrypted-online PIN ciphertext

When P1=01:

Returned online data sent from multi-interface card to processing center; see the response Message data field table below:

Table D.26 Response Message Data Fields

Data	Length (in bytes)	Remarks
Transaction result	1	00: Transaction approved; 01: Transaction refused
Script Execution Result (Label DF31)	8	TLV format
Terminal Verification Result (TVR) (Label 95)	7	TLV format
Application Transaction Code (ATC) (Label 9F36)	5	TLV format
Ciphertext Information Type (CID) (Label 9F27)	4	TLV format
Application Ciphertext (AC) (Label 9F26)	11	TLV format
Issuer Application Data (Label 9F10)	Maximum 35	TLV format

D.5.5 Response Message Status Codes

"9000" designates that this command was executed successfully.

For possible error messages sent in response to this command, see Appendix C.

Appendix E Personalization

UnionPay multi-interface cards are unique, and traditional financial IC cards have personalization methods and processes which may not be appropriate for multi-interface cards. In accordance with the characteristics of the particular multi-interface card and abiding by the current UnionPay personalization system, new personalization processes for multi-interface cards can be defined, thus strengthening their security and usability.

In accordance with the application functionality already inherent to UnionPay multi-interface cards, personalization has two parts; UICS financial application personalization and secure channel application personalization. UICS financial application personalization is completed by the issuer and uses card generation data that is generated by the issuer's data preparation system. See *UICS Auxiliary Specifications part 5-* for an in-depth explanation of financial application personalization processes. Secure channel application personalization includes at at least writing multi-interface sequence numbers, CA Root certificates, multi-interface card certificates, and PIN encryption certificates into the card.

It is recommended that multi-interface card personalization be conducted in two parts. The first part is completed by the issuer personalization center and primarily includes financial application personalization, secure channel multi-interface card sequence numbers, CA Root certificates, and PIN encryption certificate personalization; the second part connects the multi-interface card to the CA center, which then allows for the downloading of multi-interface card certificates from the center's RA.

Appendix F Audio Communication API

F.1 Java Version

F.1.1 Encapsulation

package com.mcard.*vendor*.*ver*

F.1.2 Interface

public class mcard

F.1.3 getDrvInfo

Functionality:

Return driver version, include manufacture name, version, edit date,etc.

Parameter:

Info: deriver description information.

Return:

Driver version, e.g.: 103 which means 1.03 version.

Prototype:

public int getDrvInfo(String info)

F.1.4 checkCard

Functionality：

Detect if multi-interface card exist.

Parameter:

Sysname: is application system name, used to tell and prevent different multi-interface card to be insert to same cellphone and same application.

Return:

Return = 0 means successful.

Prototype:

public boolean checkCard(byte dev[],String system)

F.1.5 open

Functionality：

Open communication interface, enter communication mode.

Parameter:

Sysname: is application system name, used to tell and prevent different multi-interface card to be insert to same cellphone and same application.

Return:

Return = 0 means successful. Smaller than 0 means error code.

Prototype:

public int open(byte dev[],String system)

F.1.6 reset

Functionality：

Reset card

Parameter:

NA

Return:

Return value >=0 means successful, meanwhile atr save card reset answer flow byte, value <0 means error.

Prototype:

public int reset(byte dev[],byte atr[])

F.1.7 apdu

Functionality：

Send command to card and get return.

Parameter:

radpu APDU answer cache

capdu APDU sending content.

Return:

Retur: value >=2, successful, value < 0 means error.

Prototype:

public int apdu(byte dev[],byte rapdu[], int rsize, byte capdu[], int clen)

F.1.8 close

Functionality：

Close card power.

Parameter:

NA

Return:

Return = 0 means successful. Smaller than 0 means error code.

Prototype:

public void close(byte dev[])

F.1.9 waitConfirm

Functionality：

Wait for pressing anti-hijack button.

Parameter：

NA

Return：

Return >= 0 means successful. Smaller than 0 means error code.

Prototype：

public int waitConfirm()

F.1.10 checkConfirm

Functionality：

Check if press anti-hijack button.

Parameter：

NA

Return：

Return = 0 means successful. Smaller than 0 means error code.

Prototype：

public int checkConfirm()

F.2 C# Version

F.2.1 Header file

#include <mcard.h>。

F.2.2 getDrvInfo

Functionality:

Return driver version, include manufacture name, version, edit date,etc.

Parameter:

Info: deriver description information.

Return:

Driver version, e.g.: 103 which means 1.03 version.

Prototype:

public int getDrvInfo(byte [] info)

F.2.3 checkCard

Functionality：

Detect if multi-interface card exist.

Parameter:

Sysname: is application system name, used to tell and prevent different multi-interface card to be insert to same cellphone and same application.

Return:

Return = 0 means successful.Prototype

Prototype:

public bool checkCard(byte [] dev,byte [] system)

F.2.4 open

Functionality：

Open communication interface, enter communication mode.

Parameter:

Sysname: is application system name, used to tell and prevent different multi-interface card to be insert to same cellphone and same application.

Return:

Return = 0 means successful. Smaller than 0 means error code.

Prototype:

public int open(byte [] dev,byte [] system)

F.2.5 reset

Functionality：

Reset card

Parameter:

NA

Return:

Return value >=0 means successful, meanwhile atr save card reset answer flow byte, value <0 means error.

Prototype:

public int reset(byte [] dev,byte [] atr)

F.2.6 apdu

Functionality：

Send command to card and get return.

Parameter:

radpu APDU answer cache

capdu APDU sending content.

Return:

Retur: value >=2, successful, value < 0 means error.

Prototype:

public int apdu(byte [] dev,byte []rapdu, int rsize, byte [] capdu, int clen)

F.2.7 close

Functionality：

Close card power.

Parameter:

NA

Return:

Return = 0 means successful. Smaller than 0 means error code.

Prototype:

public void close(byte [] dev)

F.2.8 waitConfirm

Functionality：

Wait for pressing anti-hijack button.

Parameter：

NA

Return：

Return >= 0 means successful. Smaller than 0 means error code.

Prototype：

public int waitConfirm()

F.2.9 checkConfirm

Functionality：

Check if press anti-hijack button.

Parameter：

NA

Return：

Return = 0 means successful. Smaller than 0 means error code.

Prototype：

public int checkConfirm()

F.3 Prototype Error Code Definition

Table 35 Audio Communication API Error Code Definition

Name	Value	Definition
OK	0	Success
ERR	-1	Normal Error
EPARAM	-3	Parameter Error
ENODATA	-5	No data return
EPAR	-7	Byte check error
EFRAME	-9	Character frame error
ETIMEOUT	-11	Timeout
ECHK	-13	Check error
EPACK	-15	Format error
EBUSY	-17	Busy
EFULL	-19	Cache full
EPOWERON	-21	Power failure
ENOOBJ	-23	Divide not exist
ELOWBAT	-25	Low power

Appendix GAnti-hijack Configuration

G.1 Anti-hijack Threshold value setup

Cardholder has 2 ways to configure anti-hijack threshold:

1. Cardholder use multi-interface card keyboard to set threshold

E.g.: when power on, press and hold ‘C’ key, then multi-interface card enter setting mode, cardholder use keyboard, input threshold value or choose threshold value, and press ‘OK’ to finish configuration. Threshold is digit, can’t be alphabet or other characters.

2. Cardholder use external device to input threshold value to multi-interface card and confirm

E.g.: cardholder use mobile device to input threshold, multi-interface card display value, and wait for cardholder confirm threshold value and press ‘OK’ button to finish configuration.

Table 36 Anti-hijack status

Transaction type		**Anti-hijack threshold**	**Required press button**	**Screen display content**
Multi-interface card in manufacture status		0	Required	
Contactless transaction	Credit/debit, low-value payment	Cardholder set	If transaction amount greater than threshold, require cardholder press button	Transaction amount
Internet channel transaction	Credit/debit	If threshold is 0, cardholder can’t set	Required	At least contains: transaction amount
	Digital certificate authentication	If threshold is 0, cardholder can’t set	Required	At least contains: transaction amount, receiver PAN, receiver name